Tunneling v5

Implementation code - supervised tunnel, agent bus, telemetry

BRIDGE 5.0.0

BRANCH	arena/706ae6d5-tunneling
DIFF BASE	92cde4e280cfcc947f204cf7fbf01e329f9a0282 -> HEAD
GENERATED	2026-11-03
TESTS	25 passing (python3 -B roblox-bridge/test_bridge_mcp.py)
RUNTIME	Python 3 stdlib only (no pip packages)

INSIDE

1 What this release contains · 2 File inventory · 3 How to apply and verify · 4 Complete sources (tunnel_manager, bus_core, auth_core, telemetry, seed_agent_bus) · 5 Unified diffs for every modified file · 6 Verification snapshot

1. What this release contains

v5 turns the local Studio bridge into an ops platform. The MCP tool surface grows to **70 tools** (adds `agent_bus` and `bridge_ops`) and **8 resources**. Four new stdlib modules carry the new behaviour, and the existing server/CLI/setup paths are extended without breaking v4 callers.

- **Supervised tunnel** - auto-detected cloudflared/ngrok, URL scraping, public health probes, backoff restarts, named tunnels, LAN URLs, and a hard refusal to publish a token-less bridge.
- **Agent Bus** - presence + mailbox + lease-based task board so multiple agents and humans can share one Studio session without collisions.
- **Scoped tokens** - hashed read/write/admin credentials with expiry and per-token rate limits; read tokens cannot mutate Studio over MCP or REST.
- **Resilient jobs** - idempotency keys, priorities, bounded retries, per-job timeouts and orphan requeue.
- **Observability** - Prometheus metrics, SSE event stream, SHA-256 hash-chained audit log, consolidated ops report, and dashboard tab 7.

2. File inventory

File	Kind	Purpose
`roblox-bridge/tunnel_manager.py`	new	Supervised cloudflared/ngrok tunnel
`roblox-bridge/bus_core.py`	new	Multi-agent presence, mailbox and lease-based task board
`roblox-bridge/auth_core.py`	new	Scoped tokens, expiry, revocation, token-bucket limits
`roblox-bridge/telemetry.py`	new	Metrics, SSE event stream, hash-chained audit log
`roblox-bridge/tools/seed_agent_bus.py`	new	Idempotent Agent Bus seeding (online or offline)
`roblox-bridge/server.py`	diff	HTTP bridge, auth scopes, ops endpoints, dashboard tab 7
`roblox-bridge/mcp_core.py`	diff	agent_bus + bridge_ops tools, 3 resources, client helpers
`roblox-bridge/arena_studio.py`	diff	ops / metrics / audit / tunnel / tokens / bus CLI
`roblox-bridge/setup.py`	diff	Supervised tunnel launch + URL watcher
`roblox-bridge/test_bridge_mcp.py`	diff	v5 test coverage (scopes, bus, reaping, audit, tunnel)
`roblox-bridge/mcp_server.py`	diff	Version banner
`roblox-bridge/ab.ps1`	diff	Version banner + ops examples

3. How to apply and verify

Copy the new files into `roblox-bridge/` and apply the diffs to the modified files (both sides are shown in this document). Then:

```
python3 -B roblox-bridge/test_bridge_mcp.py # expect: Ran 25 tests ... OK
python3 roblox-bridge/arena_studio.py ops # peers, queue, tunnel, agents, tasks
python3 roblox-bridge/arena_studio.py audit --verify # hash chain intact
python3 setup.py --tunnel # supervised public tunnel
```

Auth is unchanged for existing installs: `bridge.token` is the admin owner credential and nothing new is required. Runtime state (audit, tokens, bus, last tunnel URL) is written to the gitignored `roblox-bridge/bridge_state/`.

4. New source files

File	Lines	Purpose
`roblox-bridge/tunnel_manager.py`	475	Supervised cloudflared/ngrok tunnel
`roblox-bridge/bus_core.py`	752	Multi-agent presence, mailbox and lease-based task board
`roblox-bridge/auth_core.py`	273	Scoped tokens, expiry, revocation, token-bucket limits
`roblox-bridge/telemetry.py`	471	Metrics, SSE event stream, hash-chained audit log
`roblox-bridge/tools/seed_agent_bus.py`	129	Idempotent Agent Bus seeding (online or offline)

Complete sources follow first, then unified diffs for every modified file. Line numbers are left-margin references; wrapped lines carry no number.

roblox-bridge/tunnel_manager.py

475 lines

Supervised cloudflared/ngrok tunnel

```python
#!/usr/bin/env python3
"""
tunnel_manager.py - Supervised public tunnel for the Arena Roblox Studio bridge.

`python setup.py --tunnel` starts a one-shot Cloudflare quick tunnel and hopes it
stays up. This module makes the tunnel a managed service instead:

  * auto-detects `cloudflared` or `ngrok` on PATH (with install hints if absent),
  * supervises the child process with exponential backoff and automatic restart,
  * scrapes the public URL from the child's output and re-learns it after a restart,
  * probes `<publicUrl>/api/health` and marks the tunnel degraded when it stops
    answering, restarting it after repeated failures,
  * supports token-based named Cloudflare tunnels and a user-managed static URL,
  * reports LAN URLs so phones/tablets on the same Wi-Fi can reach the dashboard,
  * exposes everything as JSON for `GET /api/tunnel` and the Control Center.

Zero dependencies; works on Windows, macOS and Linux.
"""

import json
import os
import re
import shutil
import socket
import subprocess
import threading
import time
import urllib.error
import urllib.request
from collections import deque
from pathlib import Path

TUNNEL_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

CLOUDFLARE_URL_RE = re.compile(r"https://[a-z0-9][a-z0-9-]*\.trycloudflare\.com", re.I)
NGROK_URL_RE = re.compile(r"https://[a-z0-9][a-z0-9-.]*\.ngrok(?:-free)?\.(?:app|io|dev)", re.I)
ANY_URL_RE = re.compile(r"https://[a-z0-9][a-z0-9.-]+\.[a-z]{2,}(?::\d+)?", re.I)

INSTALL_HINTS = {
    "cloudflared": [
        "Windows:  winget install --id Cloudflare.cloudflared",
        "macOS:    brew install cloudflared",
        "Linux:    https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/",
    ],
    "ngrok": [
        "Windows:  winget install --id ngrok.ngrok",
        "macOS:    brew install ngrok/ngrok/ngrok",
        "Any:      https://ngrok.com/download (then `ngrok config add-authtoken <token>`)",
    ],
}


def _now() -> float:
    return time.time()


def local_ip_addresses() -> list:
    """Best-effort LAN IPv4 addresses (no packet is actually sent)."""
    found = []
```

```
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            sock.connect(("10.255.255.255", 1))
            found.append(sock.getsockname()[0])
        finally:
            sock.close()
    except OSError:
        pass
    try:
        host = socket.gethostname()
        for info in socket.getaddrinfo(host, None, socket.AF_INET):
            ip = info[4][0]
            if ip not in found and not ip.startswith("127."):
                found.append(ip)
    except OSError:
        pass
    return [ip for ip in found if not ip.startswith("127.")]


def find_tunnel_binary(preferred: str = None) -> tuple:
    """Return (flavor, path) for the first tunnel binary found, else (None, None)."""
    candidates = []
    if preferred and preferred not in ("auto", ""):
        candidates.append(preferred)
    candidates += ["cloudflared", "ngrok"]
    for name in candidates:
        found = shutil.which(name)
        if found:
            return ("cloudflared" if "cloudflared" in Path(found).name.lower() else "ngrok"), found
    return None, None


class TunnelManager:
    """Starts, watches, restarts and reports a public tunnel for the local bridge."""

    def __init__(
        self,
        port: int = 8077,
        binary: str = None,
        flavor: str = "auto",
        named_token: str = None,
        name: str = None,
        public_url: str = None,
        health_interval: float = 20.0,
        health_failures_before_restart: int = 4,
        max_backoff: float = 30.0,
        state_dir=None,
        auto_start: bool = False,
    ):
        self.port = int(port)
        self.local_url = f"http://127.0.0.1:{self.port}"
        self.requested_flavor = (flavor or "auto").lower()
        self.binary_arg = binary
        self.named_token = named_token or os.environ.get("TUNNEL_TOKEN") or None
        self.name = name
        self.user_public_url = (public_url or os.environ.get("BRIDGE_PUBLIC_URL") or "").strip() or None
        self.health_interval = float(health_interval)
        self.health_failures_before_restart = int(health_failures_before_restart)
        self.max_backoff = float(max_backoff)
```

```
        self.state_path = (Path(state_dir) if state_dir else DEFAULT_STATE_DIR) / "tunnel.json"

        self.lock = threading.RLock()
        self.stop_event = threading.Event()
        self._thread = None
        self._proc = None
        self.flavor = None
        self.binary = None
        self.state = "stopped"
        self.public_url = self.user_public_url
        self.last_error = None
        self.started_at = None
        self.online_at = None
        self.restarts = 0
        self.url_changes = 0
        self.url_changed_at = None
        self.proc_pid = None
        self.health = {"lastProbe": None, "ok": None, "latencyMs": None, "failures": 0, "consecutiveFailures": 0}
        self.log_tail = deque(maxlen=60)
        self.last_output_at = None
        self._last_health = 0.0
        self._load_last_url()
        if auto_start:
            self.start()

    # -- persistence of last known URL ---------------------------------------

    def _load_last_url(self):
        try:
            raw = json.loads(self.state_path.read_text(encoding="utf-8"))
            if not self.public_url and raw.get("publicUrl"):
                self.public_url = raw["publicUrl"]
                self.last_error = None
                self._loaded_url = True
        except (OSError, ValueError):
            pass

    def _persist_url(self):
        try:
            self.state_path.parent.mkdir(parents=True, exist_ok=True)
            self.state_path.write_text(
                json.dumps({"publicUrl": self.public_url, "saved": _now(), "version": TUNNEL_VERSION}),
                encoding="utf-8",
            )
        except OSError:
            pass

    # -- command construction ------------------------------------------------

    def _command(self) -> list:
        if not self.binary:
            raise RuntimeError("No tunnel binary available")
        if self.flavor == "cloudflared":
            if self.named_token:
                # Named tunnel: the ingress/hostname config lives in Cloudflare;
                # `--url` is only valid for quick tunnels.
                return [self.binary, "tunnel", "--no-autoupdate", "run", "--token", self.named_token]
            return [self.binary, "tunnel", "--url", self.local_url, "--no-autoupdate"]
        if self.flavor == "ngrok":
            cmd = [self.binary, "http", str(self.port), "--log", "stdout", "--log-format", "logfmt"]
```

```
            return cmd
        raise RuntimeError(f"Unsupported tunnel flavor: {self.flavor}")

    def _detect(self):
        flavor, path = find_tunnel_binary(self.binary_arg)
        if not flavor and self.requested_flavor in ("cloudflared", "ngrok"):
            # Respect an explicit preference even if the generic detection failed.
            found = shutil.which(self.requested_flavor)
            if found:
                flavor, path = self.requested_flavor, found
        with self.lock:
            self.flavor = flavor if not self.requested_flavor or self.requested_flavor == "auto" else (self.requested_flavor if path else flavor)
            self.binary = path
        return bool(path)

    def available(self) -> bool:
        return self.binary is not None or shutil.which(self.binary_arg or "") is not None

    # -- lifecycle ---------------------------------------------------------

    def start(self) -> dict:
        with self.lock:
            if self.user_public_url:
                self.state = "online"
                self.public_url = self.user_public_url
                self.online_at = self.online_at or _now()
                return self.status()
            if self._thread and self._thread.is_alive():
                return self.status()
            if not self._detect():
                self.state = "unavailable"
                self.last_error = (
                    "No tunnel binary found on PATH. Install cloudflared or ngrok, "
                    "or pass BRIDGE_PUBLIC_URL for a tunnel you run yourself."
                )
                return self.status()
            self.stop_event.clear()
            self.last_error = None
            self.state = "starting"
            self.started_at = self.started_at or _now()
            self._thread = threading.Thread(target=self._supervise, name="tunnel-supervisor", daemon=True)
            self._thread.start()
            return self.status()

    def stop(self) -> dict:
        self.stop_event.set()
        with self.lock:
            proc = self._proc
        if proc and proc.poll() is None:
            try:
                proc.terminate()
                try:
                    proc.wait(timeout=5)
                except subprocess.TimeoutExpired:
                    proc.kill()
            except OSError:
                pass
        thread = self._thread
        if thread and thread.is_alive() and thread is not threading.current_thread():
            thread.join(timeout=6)
```

```
        with self.lock:
            self._thread = None
            self._proc = None
            self.proc_pid = None
            self.state = "stopped"
            if not self.user_public_url:
                self.online_at = None
        return self.status()

    def restart(self) -> dict:
        self.stop()
        with self.lock:
            self.restarts += 1
        time.sleep(0.4)
        return self.start()

    # -- supervision -------------------------------------------------------

    def _supervise(self):
        backoff = 1.0
        while not self.stop_event.is_set():
            try:
                cmd = self._command()
            except RuntimeError as exc:
                with self.lock:
                    self.state = "error"
                    self.last_error = str(exc)
                return
            with self.lock:
                self.state = "starting"
                self._append_log_unlocked(f"$ {' '.join(cmd[:3])}{' ...' if len(cmd) > 3 else ''}")
            try:
                proc = subprocess.Popen(
                    cmd,
                    stdout=subprocess.PIPE,
                    stderr=subprocess.STDOUT,
                    text=True,
                    bufsize=1,
                    start_new_session=(os.name != "nt"),
                )
            except (OSError, ValueError) as exc:
                with self.lock:
                    self.state = "error"
                    self.last_error = f"Failed to launch {self.flavor}: {exc}"
                    self._append_log_unlocked(self.last_error)
                if self.stop_event.wait(backoff):
                    break
                backoff = min(self.max_backoff, backoff * 2)
                continue
            with self.lock:
                self._proc = proc
                self.proc_pid = proc.pid
            reader = threading.Thread(target=self._read_output, args=(proc,), daemon=True)
            reader.start()
            # Wait for the process to exit, probing health in the meantime.
            while not self.stop_event.is_set() and proc.poll() is None:
                if self.stop_event.wait(timeout=1.0):
                    break
                self._maybe_probe()
            reader.join(timeout=2)
```

```
            if self.stop_event.is_set():
                break
            code = proc.poll()
            with self.lock:
                self._proc = None
                self.proc_pid = None
                self._append_log_unlocked(f"[supervisor] {self.flavor} exited with code {code}; reconnecting...")
                self.state = "reconnecting"
                self.restarts += 1
                self.health["ok"] = False
                self.health["consecutiveFailures"] = 0
                # A dead child invalidates its quick-tunnel URL; a new one is
                # scraped from the replacement process.
                self.public_url = None
            if self.stop_event.wait(backoff):
                break
            backoff = min(self.max_backoff, backoff * 2)
        with self.lock:
            if self.state not in ("stopped",):
                self.state = "stopped"

    def _read_output(self, proc):
        stream = proc.stdout
        if not stream:
            return
        for raw_line in iter(stream.readline, ""):
            line = (raw_line or "").rstrip()
            if not line:
                continue
            with self.lock:
                self._append_log_unlocked(line)
                self.last_output_at = _now()
            match = None
            if self.flavor == "ngrok":
                match = NGROK_URL_RE.search(line) or CLOUDFLARE_URL_RE.search(line)
            else:
                match = CLOUDFLARE_URL_RE.search(line) or NGROK_URL_RE.search(line)
            if match:
                self._set_url(match.group(0))
        try:
            stream.close()
        except OSError:
            pass

    def _append_log_unlocked(self, line: str):
        self.log_tail.append(line[:400])

    def _set_url(self, url: str):
        url = url.rstrip("/")
        with self.lock:
            if self.public_url == url:
                return
            changed = self.public_url is not None
            self.public_url = url
            self.url_changed_at = _now()
            self.state = "online"
            self.online_at = _now()
            self.last_error = None
            self.health["ok"] = None
            self.health["failures"] = 0
```

```
            self.health["consecutiveFailures"] = 0
            if changed:
                self.url_changes += 1
        self._persist_url()

    def _maybe_probe(self):
        now = _now()
        if now - self._last_health < self.health_interval:
            return
        if not self.public_url:
            self._last_health = now
            if self.last_output_at and now - self.last_output_at > 45 and self.state == "starting":
                with self.lock:
                    self.last_error = "Tunnel process started but has not published a URL yet."
            return
        self._last_health = now
        threading.Thread(target=self._probe_once, name="tunnel-health", daemon=True).start()

    def _probe_once(self):
        url = f"{self.public_url}/api/health"
        started = time.monotonic()
        ok = False
        detail = None
        try:
            req = urllib.request.Request(url, headers={"User-Agent": f"arena-tunnel/{TUNNEL_VERSION}"})
            with urllib.request.urlopen(req, timeout=10) as resp:
                ok = 200 <= resp.status < 500
                detail = f"HTTP {resp.status}"
        except urllib.error.HTTPError as exc:
            ok = exc.code < 500
            detail = f"HTTP {exc.code}"
        except Exception as exc:  # noqa: BLE001 - network errors are all non-fatal
            detail = str(exc)[:200]
        latency = round((time.monotonic() - started) * 1000, 1)
        restart = False
        with self.lock:
            self.health["lastProbe"] = _now()
            self.health["latencyMs"] = latency
            self.health["ok"] = ok
            self.health["detail"] = detail
            if ok:
                self.health["failures"] = 0
                self.health["consecutiveFailures"] = 0
                if self.state == "degraded":
                    self.state = "online"
            else:
                self.health["failures"] = int(self.health.get("failures") or 0) + 1
                self.health["consecutiveFailures"] = int(self.health.get("consecutiveFailures") or 0) + 1
                if self.health["consecutiveFailures"] >= 2 and self.state == "online":
                    self.state = "degraded"
                if self.health["consecutiveFailures"] >= self.health_failures_before_restart:
                    restart = True
            if restart:
                self._append_log_unlocked("[supervisor] public URL failed repeated health probes; restarting tunnel")
        if restart:
            proc = self._proc
            if proc and proc.poll() is None:
                try:
                    proc.terminate()
                except OSError:
```

```
                    pass
            with self.lock:
                self.restarts += 1

    # -- reporting ------------------------------------------------------------

    def status(self) -> dict:
        with self.lock:
            now = _now()
            state = self.state
            if self.user_public_url and state == "stopped":
                state = "online"
            uptime = round(now - self.online_at, 1) if self.online_at and state == "online" else 0
            lan = [
                f"http://{ip}:{self.port}" for ip in local_ip_addresses()
            ]
            return {
                "version": TUNNEL_VERSION,
                "state": state,
                "running": bool(self._thread and self._thread.is_alive()) or bool(self.user_public_url),
                "flavor": self.flavor,
                "binary": self.binary,
                "binaryAvailable": bool(self.binary),
                "managed": not bool(self.user_public_url),
                "named": bool(self.named_token),
                "localUrl": self.local_url,
                "lanUrls": lan,
                "publicUrl": self.public_url,
                "mcpEndpoint": f"{self.public_url}/mcp" if self.public_url else None,
                "healthEndpoint": f"{self.public_url}/api/health" if self.public_url else None,
                "onlineSeconds": uptime,
                "restarts": self.restarts,
                "urlChanges": self.url_changes,
                "urlChangedAt": self.url_changed_at,
                "pid": self.proc_pid,
                "lastError": self.last_error,
                "health": dict(self.health),
                "logTail": list(self.log_tail)[-25:],
                "installHints": INSTALL_HINTS.get(self.flavor or "cloudflared"),
                "serverTime": now,
            }

    def handle(self, payload: dict) -> dict:
        action = ((payload or {}).get("action") or "status").lower()
        if action == "status":
            return self.status()
        if action == "start":
            return self.start()
        if action == "stop":
            return self.stop()
        if action == "restart":
            return self.restart()
        if action == "lan":
            return {"lanUrls": [f"http://{ip}:{self.port}" for ip in local_ip_addresses()], "port": self.port}
        raise ValueError("Unknown tunnel action. Use status, start, stop, restart or lan.")

```

roblox-bridge/bus_core.py

752 lines

Multi-agent presence, mailbox and lease-based task board

```python
#!/usr/bin/env python3
"""
bus_core.py - Multi-agent collaboration bus for the Arena Roblox Studio bridge.

Zero-dependency, thread-safe, JSON-persisted coordination layer that lets several
AI agents (and humans) share one Roblox Studio session without stepping on each
other:

  * Presence registry   - named agents with roles, capabilities and heartbeats.
  * Mailbox             - direct + broadcast messages, cursor-based inbox, acks.
  * Task board          - claim/lease/complete work items with priorities, tags
                          and dependencies. A lease expires automatically, so a
                          crashed agent cannot permanently hold a task.
  * Event feed          - bounded, sequence-numbered activity log.
  * Handoff export      - markdown brief for the next agent.

Everything is plain stdlib so it can be embedded in server.py, used from the CLI,
or driven through the `agent_bus` MCP tool.
"""

import json
import os
import threading
import time
import uuid
from collections import deque
from pathlib import Path

BUS_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

AGENT_STATUSES = ("online", "busy", "idle", "offline")
TASK_STATUSES = ("todo", "in_progress", "blocked", "done", "cancelled")
MESSAGE_KINDS = ("note", "question", "answer", "handoff", "alert", "result")
PRIORITIES = ("low", "normal", "high", "critical")
_PRIORITY_RANK = {"low": 3, "normal": 2, "high": 1, "critical": 0}


def _now() -> float:
    return time.time()


def _clean(text, limit: int = 8000) -> str:
    if text is None:
        return ""
    text = str(text)
    return text[:limit]


class AgentBus:
    """Presence + mailbox + task board shared by every agent on the bridge."""

    def __init__(
        self,
        path=None,
        max_messages: int = 2000,
        max_events: int = 1000,
        default_lease_seconds: int = 900,
        presence_ttl: float = 90.0,
        persist: bool = True,
```

```
    ):
        self.path = Path(path) if path else DEFAULT_STATE_DIR / "bus.json"
        self.persist = bool(persist)
        self.lock = threading.RLock()
        self.agents = {}
        self.messages = deque(maxlen=max_messages)
        self.events = deque(maxlen=max_events)
        self.tasks = {}
        self.default_lease_seconds = int(default_lease_seconds)
        self.presence_ttl = float(presence_ttl)
        self.seq = 0
        self.created = _now()
        self._load()

    # -- persistence -----------------------------------------------------

    def _load(self):
        if not self.persist or not self.path.is_file():
            return
        try:
            raw = json.loads(self.path.read_text(encoding="utf-8"))
        except (OSError, ValueError):
            return
        if not isinstance(raw, dict):
            return
        with self.lock:
            self.agents = {a["id"]: a for a in raw.get("agents", []) if a.get("id")}
            self.messages = deque(raw.get("messages", [])[-self.messages.maxlen:], maxlen=self.messages.maxlen)
            self.events = deque(raw.get("events", [])[-self.events.maxlen:], maxlen=self.events.maxlen)
            self.tasks = {t["id"]: t for t in raw.get("tasks", []) if t.get("id")}
            self.seq = int(raw.get("seq") or 0)

    def _save(self):
        if not self.persist:
            return
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            payload = {
                "version": BUS_VERSION,
                "saved": _now(),
                "seq": self.seq,
                "agents": list(self.agents.values()),
                "messages": list(self.messages),
                "events": list(self.events),
                "tasks": list(self.tasks.values()),
            }
            tmp = self.path.with_suffix(".tmp")
            tmp.write_text(json.dumps(payload, ensure_ascii=False, separators=(",", ":")), encoding="utf-8")
            os.replace(tmp, self.path)
        except OSError:
            pass

    # -- internals -------------------------------------------------------

    def _next_seq_unlocked(self) -> int:
        self.seq += 1
        return self.seq

    def _emit_unlocked(self, kind: str, data: dict) -> dict:
        event = {"seq": self._next_seq_unlocked(), "kind": kind, "ts": _now(), "data": data}
```

```
        self.events.append(event)
        return event

    def _agent_unlocked(self, agent_id: str) -> dict:
        agent = self.agents.get(agent_id)
        if not agent:
            raise KeyError(f"Unknown agent id: {agent_id!r}")
        return agent

    def _touch_unlocked(self, agent: dict):
        agent["lastSeen"] = _now()
        agent["online"] = True

    def _expire_leases_unlocked(self):
        now = _now()
        for task in self.tasks.values():
            lease = task.get("lease") or {}
            if task.get("status") == "in_progress" and lease.get("expiresAt") and lease["expiresAt"] < now:
                prev_owner = task.get("assignee")
                task["status"] = "todo"
                task["assignee"] = None
                task["lease"] = None
                task["updated"] = now
                task["leaseExpiries"] = int(task.get("leaseExpiries", 0)) + 1
                self._emit_unlocked("task.lease_expired", {"taskId": task["id"], "previousAssignee": prev_owner})

    # -- agents / presence ---------------------------------------------------

    def register_agent(
        self,
        name: str,
        role: str = "agent",
        capabilities=None,
        agent_id: str = None,
        ttl: float = None,
        meta: dict = None,
    ) -> dict:
        name = (name or "").strip()
        if not name:
            raise ValueError("name is required")
        with self.lock:
            # Reuse an existing registration with the same name (case-insensitive).
            existing = next((a for a in self.agents.values() if a["name"].lower() == name.lower()), None)
            now = _now()
            caps = [str(c) for c in (capabilities or [])][:32]
            if existing:
                existing.update({
                    "name": name,
                    "role": _clean(role, 40) or "agent",
                    "capabilities": caps or existing.get("capabilities", []),
                    "status": "online",
                    "online": True,
                    "lastSeen": now,
                    "ttl": float(ttl or self.presence_ttl),
                    "meta": meta if isinstance(meta, dict) else existing.get("meta", {}),
                })
                agent = existing
                created_new = False
            else:
                aid = agent_id or ("agent-" + uuid.uuid4().hex[:8])
```

```
                if aid in self.agents:
                    aid = "agent-" + uuid.uuid4().hex[:8]
                agent = {
                    "id": aid,
                    "name": name,
                    "role": _clean(role, 40) or "agent",
                    "capabilities": caps,
                    "status": "online",
                    "online": True,
                    "firstSeen": now,
                    "lastSeen": now,
                    "ttl": float(ttl or self.presence_ttl),
                    "meta": meta if isinstance(meta, dict) else {},
                }
                self.agents[agent["id"]] = agent
                created_new = True
            self._emit_unlocked("agent.register" if created_new else "agent.heartbeat", {
                "agentId": agent["id"], "name": agent["name"], "role": agent["role"],
            })
            self._save()
            return self._agent_view_unlocked(agent)

    def heartbeat(self, agent_id: str, status: str = None) -> dict:
        with self.lock:
            agent = self._agent_unlocked(agent_id)
            self._touch_unlocked(agent)
            if status in AGENT_STATUSES:
                agent["status"] = status
            self._save()
            return self._agent_view_unlocked(agent)

    def set_status(self, agent_id: str, status: str) -> dict:
        if status not in AGENT_STATUSES:
            raise ValueError(f"status must be one of {', '.join(AGENT_STATUSES)}")
        with self.lock:
            agent = self._agent_unlocked(agent_id)
            self._touch_unlocked(agent)
            agent["status"] = status
            self._save()
            return self._agent_view_unlocked(agent)

    def _agent_view_unlocked(self, agent: dict) -> dict:
        row = dict(agent)
        age = _now() - float(agent.get("lastSeen") or 0)
        row["ageSeconds"] = round(age, 1)
        row["online"] = age <= float(agent.get("ttl") or self.presence_ttl)
        if not row["online"]:
            row["status"] = "offline"
        row["openTasks"] = sum(
            1 for t in self.tasks.values()
            if t.get("assignee") == agent["id"] and t.get("status") in ("todo", "in_progress", "blocked")
        )
        row["inboxUnread"] = sum(
            1 for m in self.messages
            if (m.get("to") in (agent["id"], "all", None)) and agent["id"] not in (m.get("readBy") or {})
        )
        return row

    def list_agents(self, include_offline: bool = True) -> dict:
        with self.lock:
```

```
            rows = [self._agent_view_unlocked(a) for a in self.agents.values()]
        if not include_offline:
            rows = [r for r in rows if r["online"]]
        rows.sort(key=lambda r: (not r["online"], r["name"].lower()))
        return {"agents": rows, "count": len(rows), "onlineCount": sum(1 for r in rows if r["online"])}

    # -- mailbox ------------------------------------------------------------

    def post_message(
        self,
        sender: str,
        body: str,
        to: str = None,
        subject: str = "",
        kind: str = "note",
        priority: str = "normal",
        thread_id: str = None,
        meta: dict = None,
    ) -> dict:
        sender = (sender or "").strip()
        if not sender:
            raise ValueError("sender is required")
        if not (body or subject):
            raise ValueError("body or subject is required")
        if kind not in MESSAGE_KINDS:
            kind = "note"
        if priority not in PRIORITIES:
            priority = "normal"
        with self.lock:
            # Accept an agent name as well as an id; "all" / None means broadcast.
            to_id = None
            if to and to not in ("all", "*", "broadcast"):
                to_id = to
                if to_id not in self.agents:
                    match = next((a["id"] for a in self.agents.values() if a["name"].lower() == str(to).lower()), None)
                    to_id = match or to
            msg = {
                "id": "msg-" + uuid.uuid4().hex[:10],
                "seq": self._next_seq_unlocked(),
                "ts": _now(),
                "from": sender,
                "fromId": next((a["id"] for a in self.agents.values() if a["name"].lower() == sender.lower()), None),
                "to": to_id,
                "subject": _clean(subject, 300),
                "body": _clean(body),
                "kind": kind,
                "priority": priority,
                "threadId": thread_id,
                "meta": meta if isinstance(meta, dict) else {},
                "readBy": {},
            }
            self.messages.append(msg)
            self._emit_unlocked("message.post", {
                "id": msg["id"], "from": sender, "to": to_id or "broadcast",
                "subject": msg["subject"], "kind": kind, "priority": priority,
            })
            self._save()
            return msg

    def inbox(
```

```
        self,
        agent_id: str,
        since: int = 0,
        limit: int = 50,
        unread_only: bool = False,
        include_broadcast: bool = True,
    ) -> dict:
        limit = max(1, min(int(limit or 50), 500))
        with self.lock:
            agent = self.agents.get(agent_id)
            names = {agent_id}
            if agent:
                names.add(agent["name"].lower())
            rows = []
            for msg in reversed(self.messages):
                if since and msg["seq"] <= since:
                    break
                target = msg.get("to")
                if target is None:
                    if not include_broadcast:
                        continue
                elif target not in names and str(target).lower() not in names:
                    continue
                if msg.get("from") == agent_id:
                    continue
                if unread_only and agent_id in (msg.get("readBy") or {}):
                    continue
                rows.append(msg)
                if len(rows) >= limit:
                    break
            rows.reverse()
            unread = sum(
                1 for m in self.messages
                if (m.get("to") is None or m.get("to") in names or str(m.get("to", "")).lower() in names)
                and agent_id not in (m.get("readBy") or {}) and m.get("from") != agent_id
            )
            return {
                "agentId": agent_id,
                "messages": rows,
                "count": len(rows),
                "unread": unread,
                "cursor": rows[-1]["seq"] if rows else since,
            }

    def read_thread(self, thread_id: str) -> dict:
        with self.lock:
            rows = [m for m in self.messages if m.get("threadId") == thread_id or m["id"] == thread_id]
        return {"threadId": thread_id, "messages": rows, "count": len(rows)}

    def mark_read(self, agent_id: str, message_ids=None, all_messages: bool = False) -> dict:
        now = _now()
        marked = 0
        with self.lock:
            for msg in self.messages:
                if not all_messages and message_ids and msg["id"] not in message_ids:
                    continue
                if all_messages or message_ids:
                    if agent_id not in (msg.get("readBy") or {}):
                        msg.setdefault("readBy", {})[agent_id] = now
                        marked += 1
```

```
            self._save()
        return {"agentId": agent_id, "marked": marked}

    # -- task board -----------------------------------------------------------

    def create_task(
        self,
        title: str,
        detail: str = "",
        created_by: str = "",
        assignee: str = None,
        priority: str = "normal",
        tags=None,
        depends_on=None,
        estimate_minutes: int = None,
    ) -> dict:
        title = (title or "").strip()
        if not title:
            raise ValueError("title is required")
        if priority not in PRIORITIES:
            priority = "normal"
        with self.lock:
            now = _now()
            task = {
                "id": "task-" + uuid.uuid4().hex[:8],
                "title": _clean(title, 300),
                "detail": _clean(detail),
                "createdBy": created_by or "unknown",
                "assignee": assignee,
                "status": "todo",
                "priority": priority,
                "tags": [str(t)[:40] for t in (tags or [])][:20],
                "dependsOn": [str(d) for d in (depends_on or [])][:20],
                "estimateMinutes": int(estimate_minutes) if estimate_minutes else None,
                "created": now,
                "updated": now,
                "lease": None,
                "result": None,
                "artifacts": [],
                "history": [{"ts": now, "actor": created_by or "unknown", "action": "created"}],
            }
            self.tasks[task["id"]] = task
            self._emit_unlocked("task.created", {"taskId": task["id"], "title": task["title"], "priority": priority})
            self._save()
            return dict(task)

    def list_tasks(self, status: str = None, assignee: str = None, tag: str = None, include_done: bool = True) -> dict:
        with self.lock:
            self._expire_leases_unlocked()
            rows = list(self.tasks.values())
        if status:
            wanted = {s.strip() for s in status.split(",") if s.strip()}
            rows = [t for t in rows if t["status"] in wanted]
        if not include_done:
            rows = [t for t in rows if t["status"] not in ("done", "cancelled")]
        if assignee:
            rows = [t for t in rows if t.get("assignee") == assignee or t.get("createdBy") == assignee]
        if tag:
            rows = [t for t in rows if tag in (t.get("tags") or [])]
        rows.sort(key=lambda t: (
```

```
            {"in_progress": 0, "blocked": 1, "todo": 2, "done": 3, "cancelled": 4}.get(t["status"], 5),
            _PRIORITY_RANK.get(t.get("priority"), 2),
            t.get("created", 0),
        ))
        now = _now()
        out = []
        for t in rows:
            row = dict(t)
            lease = t.get("lease")
            row["leaseRemainingSeconds"] = round(lease["expiresAt"] - now, 1) if lease and lease.get("expiresAt") else None
            row["blockedBy"] = [d for d in (t.get("dependsOn") or []) if self.tasks.get(d, {}).get("status") != "done"]
            out.append(row)
        return {
            "tasks": out,
            "count": len(out),
            "counts": {
                s: sum(1 for t in self.tasks.values() if t["status"] == s) for s in TASK_STATUSES
            },
            "serverTime": now,
        }

    def _resolve_agent_unlocked(self, agent_id: str) -> str:
        if agent_id in self.agents:
            return agent_id
        match = next((a["id"] for a in self.agents.values() if a["name"].lower() == str(agent_id).lower()), None)
        if not match:
            raise KeyError(f"Unknown agent: {agent_id!r} - register first via action=register")
        return match

    def claim_task(self, task_id: str, agent_id: str, lease_seconds: int = None) -> dict:
        with self.lock:
            self._expire_leases_unlocked()
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            agent_id = self._resolve_agent_unlocked(agent_id)
            if task["status"] in ("done", "cancelled"):
                raise ValueError(f"Task {task_id} is already {task['status']}")
            blocked = [d for d in (task.get("dependsOn") or []) if self.tasks.get(d, {}).get("status") != "done"]
            if blocked:
                raise ValueError(f"Task {task_id} is blocked by incomplete dependencies: {', '.join(blocked)}")
            current = task.get("assignee")
            if current and current != agent_id and task.get("lease"):
                raise ValueError(f"Task {task_id} is already claimed by {current} until the lease expires")
            now = _now()
            lease_s = int(lease_seconds or self.default_lease_seconds)
            task.update({
                "assignee": agent_id,
                "status": "in_progress",
                "updated": now,
                "lease": {"owner": agent_id, "claimedAt": now, "expiresAt": now + lease_s, "seconds": lease_s},
            })
            task.setdefault("history", []).append({"ts": now, "actor": agent_id, "action": "claimed"})
            self._emit_unlocked("task.claimed", {"taskId": task_id, "agentId": agent_id, "leaseSeconds": lease_s})
            self._save()
            return dict(task)

    def renew_task(self, task_id: str, agent_id: str, lease_seconds: int = None) -> dict:
        with self.lock:
            task = self.tasks.get(task_id)
```

```
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            lease = task.get("lease") or {}
            if lease.get("owner") != agent_id:
                raise ValueError("Only the current lease owner can renew this task")
            lease_s = int(lease_seconds or lease.get("seconds") or self.default_lease_seconds)
            lease["expiresAt"] = _now() + lease_s
            task["lease"] = lease
            task["updated"] = _now()
            self._save()
            return dict(task)

    def release_task(self, task_id: str, agent_id: str, note: str = "") -> dict:
        with self.lock:
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            task.update({"assignee": None, "status": "todo", "lease": None, "updated": _now()})
            task.setdefault("history", []).append({"ts": _now(), "actor": agent_id, "action": "released", "note": note})
            self._emit_unlocked("task.released", {"taskId": task_id, "agentId": agent_id})
            self._save()
            return dict(task)

    def update_task(self, task_id: str, actor: str = "unknown", **changes) -> dict:
        allowed = {"status", "priority", "assignee", "detail", "tags", "dependsOn", "title"}
        with self.lock:
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            if changes.get("status") and changes["status"] not in TASK_STATUSES:
                raise ValueError(f"status must be one of {', '.join(TASK_STATUSES)}")
            if changes.get("priority") and changes["priority"] not in PRIORITIES:
                raise ValueError(f"priority must be one of {', '.join(PRIORITIES)}")
            for key, value in changes.items():
                if key in allowed and value is not None:
                    if key in ("detail", "title"):
                        value = _clean(value, 300 if key == "title" else 8000)
                    task[key] = value
            task["updated"] = _now()
            task.setdefault("history", []).append({"ts": _now(), "actor": actor, "action": "updated", "changes": sorted(changes)})
            self._emit_unlocked("task.updated", {"taskId": task_id, "actor": actor, "changes": sorted(changes)})
            self._save()
            return dict(task)

    def complete_task(self, task_id: str, agent_id: str, result: str = "", artifacts=None, status: str = "done") -> dict:
        if status not in ("done", "cancelled"):
            raise ValueError("complete status must be 'done' or 'cancelled'")
        with self.lock:
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            lease_owner = (task.get("lease") or {}).get("owner")
            if lease_owner and lease_owner != agent_id and task.get("assignee") not in (None, agent_id):
                raise ValueError(f"Task {task_id} is leased to {lease_owner}")
            now = _now()
            task.update({
                "status": status,
                "assignee": task.get("assignee") or agent_id,
                "result": _clean(result),
                "artifacts": [str(a)[:500] for a in (artifacts or [])][:50],
```

```
                "lease": None,
                "updated": now,
                "completedAt": now,
            })
            task.setdefault("history", []).append({"ts": now, "actor": agent_id, "action": status})
            self._emit_unlocked(f"task.{status}", {"taskId": task_id, "agentId": agent_id, "title": task["title"]})
            self._save()
            return dict(task)

    def delete_task(self, task_id: str, actor: str = "unknown") -> dict:
        with self.lock:
            task = self.tasks.pop(task_id, None)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            self._emit_unlocked("task.deleted", {"taskId": task_id, "actor": actor})
            self._save()
            return {"deleted": task_id}

    # -- feed / export ---------------------------------------------------

    def feed(self, since: int = 0, limit: int = 100) -> dict:
        limit = max(1, min(int(limit or 100), 500))
        with self.lock:
            rows = [e for e in self.events if e["seq"] > since][-limit:]
            seq = self.seq
        return {"events": rows, "count": len(rows), "cursor": seq}

    def export_handoff(self, path: str = None, include_done: bool = True) -> dict:
        """Render a markdown brief that any next agent (or human) can pick up."""
        with self.lock:
            agents = [self._agent_view_unlocked(a) for a in self.agents.values()]
            board = self.list_tasks(include_done=include_done)
            messages = list(self.messages)[-25:]
            stats = self.stats()

        lines = [
            "# Agent handoff brief",
            "",
            f"Generated: {time.strftime('%Y-%m-%d %H:%M:%S')}  ·  bus v{BUS_VERSION}",
            "",
            "## Roster",
            "",
        ]
        if agents:
            lines += ["| Agent | Role | Status | Open tasks | Capabilities |", "|---|---|---|---|---|"]
            for a in agents:
                lines.append(
                    f"| {a['name']} (`{a['id']}`) | {a['role']} | {a['status']} | {a['openTasks']} | "
                    f"{', '.join(a.get('capabilities') or []) or '-'} |"
                )
        else:
            lines.append("_No agents registered yet._")

        lines += ["", "## Task board", ""]
        if board["tasks"]:
            for t in board["tasks"]:
                mark = {"done": "x"}.get(t["status"], " ")
                owner = f" - @{t['assignee']}" if t.get("assignee") else ""
                lines.append(f"- [{mark}] **{t['title']}** ({t['status']}, {t['priority']}){owner}  ")
                if t.get("detail"):
```

```
                    lines.append(f"  {t['detail']}")
                if t.get("result"):
                    lines.append(f"  _Result:_ {t['result']}")
        else:
            lines.append("_Task board is empty._")

        lines += ["", "## Recent messages", ""]
        if messages:
            for m in messages:
                target = m.get("to") or "broadcast"
                lines.append(
                    f"- **{m['from']} -> {target}** [{m['kind']}/{m['priority']}] "
                    f"{m.get('subject') or '(no subject)'}: {m.get('body', '')[:400]}"
                )
        else:
            lines.append("_No messages._")

        lines += [
            "",
            "## Live bus stats",
            "",
            f"- agents: {stats['agents']} ({stats['agentsOnline']} online)",
            f"- tasks: {stats['tasks']} total · " + ", ".join(f"{k}={v}" for k, v in stats["taskCounts"].items()),
            f"- messages: {stats['messages']}",
            "",
            "Pick a task with `agent_bus {action:'task_claim', taskId:'...', agentId:'...'}`, "
            "or post status with `agent_bus {action:'post', ...}`.",
            "",
        ]
        text = "\n".join(lines)
        target = None
        if path:
            target = Path(path)
            target.parent.mkdir(parents=True, exist_ok=True)
            target.write_text(text, encoding="utf-8")
        return {"markdown": text, "path": str(target) if target else None, "bytes": len(text.encode("utf-8"))}

    def stats(self) -> dict:
        with self.lock:
            self._expire_leases_unlocked()
            agents = [self._agent_view_unlocked(a) for a in self.agents.values()]
            return {
                "version": BUS_VERSION,
                "agents": len(agents),
                "agentsOnline": sum(1 for a in agents if a["online"]),
                "tasks": len(self.tasks),
                "taskCounts": {s: sum(1 for t in self.tasks.values() if t["status"] == s) for s in TASK_STATUSES},
                "messages": len(self.messages),
                "events": len(self.events),
                "cursor": self.seq,
                "unreadTotal": sum(a["inboxUnread"] for a in agents),
                "serverTime": _now(),
            }

    # -- single entry point used by the HTTP + MCP layers ---------------------

    def handle(self, payload: dict) -> dict:
        action = (payload or {}).get("action", "stats")
        p = payload or {}
        if action == "register":
```

```
            return self.register_agent(
                name=p.get("name") or p.get("agent") or "agent",
                role=p.get("role") or "agent",
                capabilities=p.get("capabilities"),
                agent_id=p.get("agentId"),
                ttl=p.get("ttl"),
                meta=p.get("meta"),
            )
        if action == "heartbeat":
            return self.heartbeat(p.get("agentId") or p.get("agent"), p.get("status"))
        if action == "status":
            if p.get("agentId") and p.get("status"):
                return self.set_status(p["agentId"], p["status"])
            return self.stats()
        if action in ("agents", "list_agents"):
            return self.list_agents(bool(p.get("includeOffline", True)))
        if action in ("post", "send"):
            return self.post_message(
                sender=p.get("agentId") or p.get("from") or p.get("agent"),
                body=p.get("body") or p.get("message"),
                to=p.get("to"),
                subject=p.get("subject"),
                kind=p.get("kind") or "note",
                priority=p.get("priority") or "normal",
                thread_id=p.get("threadId"),
                meta=p.get("meta"),
            )
        if action == "inbox":
            return self.inbox(
                p.get("agentId") or p.get("agent"),
                since=int(p.get("since") or 0),
                limit=int(p.get("limit") or 50),
                unread_only=bool(p.get("unreadOnly")),
                include_broadcast=p.get("includeBroadcast", True),
            )
        if action == "thread":
            return self.read_thread(p.get("threadId") or p.get("id"))
        if action == "mark_read":
            return self.mark_read(
                p.get("agentId") or p.get("agent"),
                p.get("messageIds"),
                bool(p.get("all")),
            )
        if action in ("tasks", "list_tasks"):
            return self.list_tasks(
                p.get("status"), p.get("assignee"), p.get("tag"),
                include_done=p.get("includeDone", True),
            )
        if action == "task_create":
            return self.create_task(
                title=p.get("title"),
                detail=p.get("detail"),
                created_by=p.get("agentId") or p.get("createdBy") or "unknown",
                assignee=p.get("assignee"),
                priority=p.get("priority") or "normal",
                tags=p.get("tags"),
                depends_on=p.get("dependsOn"),
                estimate_minutes=p.get("estimateMinutes"),
            )
        if action == "task_claim":
```

```
            return self.claim_task(p.get("taskId") or p.get("id"), p.get("agentId") or p.get("agent"), p.get("leaseSeconds"))
        if action == "task_renew":
            return self.renew_task(p.get("taskId") or p.get("id"), p.get("agentId") or p.get("agent"), p.get("leaseSeconds"))
        if action == "task_release":
            return self.release_task(p.get("taskId") or p.get("id"), p.get("agentId") or p.get("agent"), p.get("note") or "")
        if action == "task_update":
            return self.update_task(
                p.get("taskId") or p.get("id"),
                actor=p.get("agentId") or p.get("actor") or "unknown",
                **{k: p.get(k) for k in ("status", "priority", "assignee", "detail", "tags", "dependsOn", "title") if k in p},
            )
        if action == "task_complete":
            return self.complete_task(
                p.get("taskId") or p.get("id"),
                p.get("agentId") or p.get("agent"),
                result=p.get("result") or "",
                artifacts=p.get("artifacts"),
                status=p.get("status") or "done",
            )
        if action == "task_delete":
            return self.delete_task(p.get("taskId") or p.get("id"), p.get("actor") or "unknown")
        if action in ("feed", "events"):
            return self.feed(int(p.get("since") or 0), int(p.get("limit") or 100))
        if action in ("export", "handoff"):
            return self.export_handoff(p.get("path"), bool(p.get("includeDone", True)))
        if action == "stats":
            return self.stats()
        raise ValueError(
            "Unknown agent_bus action. Use one of: register, heartbeat, status, agents, post, inbox, "
            "thread, mark_read, tasks, task_create, task_claim, task_renew, task_release, task_update, "
            "task_complete, task_delete, feed, export, stats."
        )

```

roblox-bridge/auth_core.py

273 lines

Scoped tokens, expiry, revocation, token-bucket limits

```python
#!/usr/bin/env python3
"""
auth_core.py - Scoped access tokens and per-token rate limiting for the bridge.

The single `bridge.token` you already have keeps working unchanged: it is treated
as the full-privilege owner credential. On top of that you can mint named,
expiring, scoped tokens so that:

  * a read-only inspector agent cannot mutate a place,
  * a teammate's CI can push scripts but cannot mint tokens or start tunnels,
  * a demo viewer can watch telemetry without touching Studio,
  * a runaway agent gets throttled instead of flooding the job queue.

Only token *hashes* are stored, and only the first 8 characters of a token are
kept for display. Tokens are shown in full exactly once, when minted.

Zero dependencies.
"""

import hashlib
import json
import os
import secrets
import threading
import time
from pathlib import Path

AUTH_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

ROLE_RANK = {"read": 1, "write": 2, "admin": 3}
ROLE_DESCRIPTIONS = {
    "read": "Inspect only: state, peers, survey, script reads, telemetry.",
    "write": "Everything read can do, plus Studio mutations, jobs and bus writes.",
    "admin": "Full control: token management, tunnel control, queue clearing, audit.",
}

# Paths that require a specific role regardless of the HTTP method.
READ_ONLY_PATHS = {
    "/api/health", "/api/state", "/api/peers", "/api/jobs", "/api/agent-guide",
    "/api/metrics", "/api/audit", "/api/events", "/api/ops", "/api/self-test",
}
ADMIN_PATHS = {"/api/tokens", "/api/audit/verify"}


def _now() -> float:
    return time.time()


def _hash(raw: str) -> str:
    return hashlib.sha256(raw.encode("utf-8")).hexdigest()


class TokenRecord:
    __slots__ = ("id", "label", "role", "created", "expires", "revoked", "last_used",
                 "uses", "rate_limit", "burst", "bucket", "bucket_ts", "token_hash", "prefix")

    def __init__(self, **kw):
        for slot in self.__slots__:
            setattr(self, slot, kw.get(slot))
```

```
61
62      def view(self) -> dict:
63          return {
64              "id": self.id,
65              "label": self.label,
66              "role": self.role,
67              "prefix": self.prefix,
68              "created": self.created,
69              "expires": self.expires,
70              "revoked": bool(self.revoked),
71              "lastUsed": self.last_used,
72              "uses": self.uses,
73              "rateLimitPerSecond": self.rate_limit,
74              "burst": self.burst,
75          }
76
77      def active(self, now: float = None) -> bool:
78          now = now or _now()
79          if self.revoked:
80              return False
81          if self.expires and self.expires < now:
82              return False
83          return True
84
85
86  class TokenStore:
87      """Hashed token registry with role scopes, expiry, revocation and rate limits."""
88
89      def __init__(self, path=None, primary_token: str = "", persist: bool = True, default_rate: float = 20.0):
90          self.path = Path(path) if path else DEFAULT_STATE_DIR / "tokens.json"
91          self.persist = bool(persist)
92          self.lock = threading.RLock()
93          self.default_rate = float(default_rate)
94          self.records = {}       # token id -> TokenRecord
95          self.by_hash = {}       # sha256(token) -> token id
96          self.primary = None     # TokenRecord for the bridge.token owner credential
97          if primary_token:
98              self.primary = TokenRecord(
99                  id="primary", label="owner (bridge.token)", role="admin",
100                 created=_now(), expires=None, revoked=False, last_used=None, uses=0,
101                 rate_limit=0.0, burst=0, bucket=0.0, bucket_ts=_now(),
102                 token_hash=_hash(primary_token), prefix=primary_token[:8],
103             )
104             self.by_hash[self.primary.token_hash] = "primary"
105         self._load()
106
107     # -- persistence ------------------------------------------------------
108
109     def _load(self):
110         if not self.persist or not self.path.is_file():
111             return
112         try:
113             raw = json.loads(self.path.read_text(encoding="utf-8"))
114         except (OSError, ValueError):
115             return
116         for row in raw.get("tokens", []):
117             try:
118                 rec = TokenRecord(
119                     id=row["id"], label=row.get("label") or "", role=row.get("role") or "read",
120                     created=row.get("created") or _now(), expires=row.get("expires"),
```

```
                    revoked=bool(row.get("revoked")), last_used=row.get("lastUsed"), uses=int(row.get("uses") or 0),
                    rate_limit=float(row.get("rateLimitPerSecond") or 0.0), burst=float(row.get("burst") or 0.0),
                    bucket=float(row.get("burst") or 0.0), bucket_ts=_now(),
                    token_hash=row["hash"], prefix=row.get("prefix") or "",
                )
            except (KeyError, TypeError, ValueError):
                continue
            self.records[rec.id] = rec
            self.by_hash[rec.token_hash] = rec.id

    def _save(self):
        if not self.persist:
            return
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            payload = {
                "version": AUTH_VERSION,
                "saved": _now(),
                "tokens": [
                    dict(r.view(), hash=r.token_hash)
                    for r in self.records.values()
                ],
            }
            tmp = self.path.with_suffix(".tmp")
            tmp.write_text(json.dumps(payload, ensure_ascii=False, separators=(",", ":")), encoding="utf-8")
            tmp.chmod(0o600)
            os.replace(tmp, self.path)
        except OSError:
            pass

    # -- authentication ------------------------------------------------------

    def authenticate(self, raw: str, consume_rate: bool = True):
        """Return (record, rate_info) or (None, None) when the token is invalid."""
        if not raw:
            return None, None
        token_hash = _hash(raw)
        with self.lock:
            rec_id = self.by_hash.get(token_hash)
            if not rec_id:
                return None, None
            rec = self.primary if rec_id == "primary" else self.records.get(rec_id)
            if not rec or not rec.active():
                return None, None
            rate_info = self._rate_check_unlocked(rec) if consume_rate else {"allowed": True, "retryAfter": 0}
            if not rate_info["allowed"]:
                return rec, rate_info
            rec.last_used = _now()
            rec.uses = int(rec.uses or 0) + 1
            return rec, rate_info

    def _rate_check_unlocked(self, rec: TokenRecord) -> dict:
        rate = float(rec.rate_limit or 0.0)
        burst = float(rec.burst or rate * 2 or 0.0)
        if rate <= 0:
            return {"allowed": True, "retryAfter": 0, "remaining": None, "limit": 0}
        now = _now()
        elapsed = max(0.0, now - float(rec.bucket_ts or now))
        tokens = min(burst, float(rec.bucket or 0.0) + elapsed * rate)
        if tokens < 1.0:
```

```
            deficit = 1.0 - tokens
            retry = max(1, int(deficit / rate) + 1)
            rec.bucket = tokens
            rec.bucket_ts = now
            return {"allowed": False, "retryAfter": retry, "remaining": 0, "limit": rate}
        rec.bucket = tokens - 1.0
        rec.bucket_ts = now
        return {"allowed": True, "retryAfter": 0, "remaining": int(rec.bucket), "limit": rate}

    # -- management ------------------------------------------------------------

    def create(self, label: str, role: str = "write", ttl_seconds: int = None, rate_limit: float = None, burst: float = None) -> dict:
        role = (role or "write").lower()
        if role not in ROLE_RANK:
            raise ValueError(f"role must be one of: {', '.join(ROLE_RANK)}")
        raw = "arb_" + secrets.token_urlsafe(30)
        rate = self.default_rate if rate_limit is None else float(rate_limit)
        with self.lock:
            rec = TokenRecord(
                id="tok-" + secrets.token_hex(6),
                label=(label or "unnamed")[:120],
                role=role,
                created=_now(),
                expires=(_now() + int(ttl_seconds)) if ttl_seconds else None,
                revoked=False,
                last_used=None,
                uses=0,
                rate_limit=rate,
                burst=float(burst if burst is not None else max(rate * 2, 1.0)),
                bucket=float(burst if burst is not None else max(rate * 2, 1.0)),
                bucket_ts=_now(),
                token_hash=_hash(raw),
                prefix=raw[:8],
            )
            self.records[rec.id] = rec
            self.by_hash[rec.token_hash] = rec.id
            self._save()
            view = rec.view()
            view["token"] = raw
            view["note"] = "Store this token now - only its hash and prefix are retained."
            return view

    def revoke(self, token_id: str) -> dict:
        with self.lock:
            if token_id == "primary":
                raise ValueError("The owner credential (bridge.token) cannot be revoked here; rotate it instead.")
            rec = self.records.get(token_id)
            if not rec:
                raise KeyError(f"Unknown token id: {token_id!r}")
            rec.revoked = True
            self._save()
            return rec.view()

    def list(self) -> dict:
        with self.lock:
            rows = []
            if self.primary:
                rows.append(self.primary.view())
            rows += [r.view() for r in self.records.values()]
        rows.sort(key=lambda r: (r["revoked"], r["created"] or 0))
```

```
        return {
            "tokens": rows,
            "count": len(rows),
            "active": sum(1 for r in rows if not r["revoked"] and (not r["expires"] or r["expires"] > _now())),
            "roles": ROLE_DESCRIPTIONS,
        }

    def owner_token(self) -> str:
        """Only used by tests/first-run bootstrap; never exposed over HTTP."""
        return ""

    @staticmethod
    def role_allows(role: str, required: str) -> bool:
        return ROLE_RANK.get(role or "read", 0) >= ROLE_RANK.get(required or "read", 0)

    def handle(self, payload: dict) -> dict:
        action = (payload or {}).get("action", "list")
        p = payload or {}
        if action == "list":
            return self.list()
        if action == "create":
            return self.create(
                p.get("label") or "unnamed",
                p.get("role") or "write",
                p.get("ttlSeconds"),
                p.get("rateLimitPerSecond"),
                p.get("burst"),
            )
        if action == "revoke":
            return self.revoke(p.get("id") or p.get("tokenId"))
        if action == "roles":
            return {"roles": ROLE_DESCRIPTIONS, "rank": ROLE_RANK}
        raise ValueError("Unknown token action. Use list, create, revoke or roles.")

```

roblox-bridge/telemetry.py

471 lines

Metrics, SSE event stream, hash-chained audit log

```python
#!/usr/bin/env python3
"""
telemetry.py - Metrics, live event stream, and tamper-evident audit log for the
Arena Roblox Studio bridge.

Zero-dependency. Three cooperating pieces:

  Metrics      counters / gauges / latency histograms with label sets, plus a
               Prometheus text exposition and a JSON snapshot for the dashboard.
  EventStream  bounded, sequence-numbered event ring with condition-variable
               waits, powering `GET /api/events` (Server-Sent Events).
  AuditLog     append-only JSONL log where every record chains the SHA-256 hash
               of its predecessor, so silent edits/tampering are detectable with
               `verify()`.

All three are thread-safe and safe to embed in server.py.
"""

import hashlib
import json
import os
import threading
import time
from collections import deque
from pathlib import Path

TELEMETRY_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

# Bounded sample count per histogram key; percentiles are computed from this
# reservoir and the true count/sum/min/max are kept exactly.
_HIST_SAMPLES = 2048


def _now() -> float:
    return time.time()


def _file_lock(fh):
    """Best-effort advisory exclusive lock for cross-process audit appends."""
    try:
        import fcntl
        fcntl.flock(fh.fileno(), fcntl.LOCK_EX)
        return "fcntl"
    except (ImportError, OSError):
        try:
            import msvcrt
            fh.seek(0)
            msvcrt.locking(fh.fileno(), msvcrt.LK_LOCK, 1)
            return "msvcrt"
        except (ImportError, OSError):
            return None


def _file_unlock(fh, kind):
    try:
        if kind == "fcntl":
            import fcntl
            fcntl.flock(fh.fileno(), fcntl.LOCK_UN)
        elif kind == "msvcrt":
```

```
            import msvcrt
            fh.seek(0)
            msvcrt.locking(fh.fileno(), msvcrt.LK_UNLCK, 1)
    except (ImportError, OSError):
        pass


def _label_key(labels: dict) -> str:
    if not labels:
        return ""
    return ",".join(f"{k}={labels[k]}" for k in sorted(labels))


def _fmt_num(value) -> str:
    if isinstance(value, bool):
        return "1" if value else "0"
    if isinstance(value, int):
        return str(value)
    if isinstance(value, float):
        if value == int(value) and abs(value) < 1e15:
            return str(int(value))
        return f"{value:.6f}".rstrip("0").rstrip(".")
    return str(value)


class Metrics:
    """Counter/gauge/histogram registry with Prometheus + JSON output."""

    def __init__(self):
        self.lock = threading.Lock()
        self.started = _now()
        self.counters = {}   # key -> {"name","labels","value"}
        self.gauges = {}
        self.histograms = {}  # key -> {"name","labels","count","sum","min","max","samples":deque}
        self.histogram_order = {}

    def _inc_unlocked(self, name: str, labels: dict, value: float):
        key = (name, _label_key(labels))
        row = self.counters.get(key)
        if row is None:
            row = {"name": name, "labels": dict(labels or {}), "value": 0.0}
            self.counters[key] = row
        row["value"] += value
        return row

    def inc(self, name: str, value: float = 1, labels: dict = None):
        with self.lock:
            self._inc_unlocked(name, labels or {}, float(value))

    def gauge(self, name: str, value: float, labels: dict = None):
        with self.lock:
            key = (name, _label_key(labels or {}))
            self.gauges[key] = {"name": name, "labels": dict(labels or {}), "value": float(value)}

    def observe(self, name: str, value: float, labels: dict = None, buckets=None):
        value = float(value)
        with self.lock:
            key = (name, _label_key(labels or {}))
            row = self.histograms.get(key)
            if row is None:
```

```
                row = {
                    "name": name,
                    "labels": dict(labels or {}),
                    "count": 0,
                    "sum": 0.0,
                    "min": value,
                    "max": value,
                    "samples": deque(maxlen=_HIST_SAMPLES),
                    "buckets": list(buckets or ()),
                    "bucketCounts": {b: 0 for b in (buckets or ())},
                }
                self.histograms[key] = row
            row["count"] += 1
            row["sum"] += value
            row["min"] = min(row["min"], value)
            row["max"] = max(row["max"], value)
            row["samples"].append(value)
            for b in row["buckets"]:
                if value <= b:
                    row["bucketCounts"][b] += 1
            self.histogram_order[key] = self.histogram_order.pop(
                key, len(self.histogram_order)
            )
            return row

    @staticmethod
    def _percentile(sorted_samples, p: float) -> float:
        if not sorted_samples:
            return 0.0
        if len(sorted_samples) == 1:
            return float(sorted_samples[0])
        rank = (len(sorted_samples) - 1) * p
        lo = int(rank)
        hi = min(lo + 1, len(sorted_samples) - 1)
        frac = rank - lo
        return float(sorted_samples[lo] * (1 - frac) + sorted_samples[hi] * frac)

    def summarize_histogram(self, row: dict) -> dict:
        samples = sorted(row["samples"])
        return {
            "count": row["count"],
            "sum": round(row["sum"], 3),
            "min": round(row["min"], 3),
            "max": round(row["max"], 3),
            "mean": round(row["sum"] / row["count"], 3) if row["count"] else 0.0,
            "p50": round(self._percentile(samples, 0.50), 3),
            "p90": round(self._percentile(samples, 0.90), 3),
            "p95": round(self._percentile(samples, 0.95), 3),
            "p99": round(self._percentile(samples, 0.99), 3),
        }

    def snapshot(self, uptime: float = None) -> dict:
        with self.lock:
            counters = [dict(v) for v in self.counters.values()]
            gauges = [dict(v) for v in self.gauges.values()]
            histograms = []
            for row in self.histograms.values():
                entry = {"name": row["name"], "labels": row["labels"]}
                entry.update(self.summarize_histogram(row))
                histograms.append(entry)
```

```
        counters.sort(key=lambda r: (r["name"], _label_key(r["labels"])))
        gauges.sort(key=lambda r: (r["name"], _label_key(r["labels"])))
        histograms.sort(key=lambda r: r["name"])
        return {
            "version": TELEMETRY_VERSION,
            "uptimeSeconds": round(uptime if uptime is not None else (_now() - self.started), 2),
            "counters": counters,
            "gauges": gauges,
            "histograms": histograms,
        }

    def prometheus(self, extra_gauges: dict = None) -> str:
        lines = []
        with self.lock:
            counters = list(self.counters.values())
            gauges = list(self.gauges.values())
            histograms = list(self.histograms.values())
        seen_types = set()

        def _decl(name, kind):
            if name not in seen_types:
                seen_types.add(name)
                lines.append(f"# TYPE {name} {kind}")

        for row in counters:
            _decl(row["name"], "counter")
            lines.append(f"{row['name']}{self._fmt_labels(row['labels'])} {_fmt_num(row['value'])}")
        for row in gauges:
            _decl(row["name"], "gauge")
            lines.append(f"{row['name']}{self._fmt_labels(row['labels'])} {_fmt_num(row['value'])}")
        for row in histograms:
            name = row["name"]
            _decl(name, "histogram")
            labels = dict(row["labels"])
            total = row["count"] or 1
            for b, count in row["bucketCounts"].items():
                bucket_labels = dict(labels)
                bucket_labels["le"] = b
                lines.append(f"{name}_bucket{self._fmt_labels(bucket_labels)} {count}")
            bucket_labels = dict(labels)
            bucket_labels["le"] = "+Inf"
            lines.append(f"{name}_bucket{self._fmt_labels(bucket_labels)} {row['count']}")
            summary = self.summarize_histogram(row)
            lines.append(f"{name}_sum{self._fmt_labels(labels)} {_fmt_num(round(row['sum'], 3))}")
            lines.append(f"{name}_count{self._fmt_labels(labels)} {row['count']}")
            for q, key in (("0.5", "p50"), ("0.9", "p90"), ("0.95", "p95"), ("0.99", "p99")):
                q_labels = dict(labels)
                q_labels["quantile"] = q
                lines.append(f"{name}{self._fmt_labels(q_labels)} {_fmt_num(summary[key])}")
            _ = total
        for name, value in (extra_gauges or {}).items():
            try:
                _decl(name, "gauge")
                lines.append(f"{name} {_fmt_num(float(value))}")
            except (TypeError, ValueError):
                continue
        return "\n".join(lines) + "\n"

    @staticmethod
    def _fmt_labels(labels: dict) -> str:
```

```
        if not labels:
            return ""
        parts = []
        for key in sorted(labels):
            value = str(labels[key]).replace("\\", "\\\\").replace('"', '\\"').replace("\n", " ")
            parts.append(f'{key}="{value}"')
        return "{" + ",".join(parts) + "}"


class EventStream:
    """Sequence-numbered event ring with blocking tail reads (SSE friendly)."""

    def __init__(self, max_events: int = 1500):
        self.lock = threading.Lock()
        self.cv = threading.Condition(self.lock)
        self.events = deque(maxlen=max_events)
        self.seq = 0

    def emit(self, kind: str, data: dict = None, level: str = "info") -> dict:
        with self.cv:
            self.seq += 1
            event = {
                "seq": self.seq,
                "ts": _now(),
                "kind": kind,
                "level": level,
                "data": data or {},
            }
            self.events.append(event)
            self.cv.notify_all()
            return event

    def since(self, since: int = 0, limit: int = 200, kinds=None) -> dict:
        limit = max(1, min(int(limit or 200), 1000))
        with self.lock:
            rows = [e for e in self.events if e["seq"] > since]
            if kinds:
                wanted = {k.strip() for k in str(kinds).split(",") if k.strip()}
                rows = [e for e in rows if any(e["kind"].startswith(w) for w in wanted)]
            rows = rows[-limit:]
            cursor = self.seq
        return {"events": rows, "count": len(rows), "cursor": cursor}

    def tail(self, cursor: int, timeout: float = 15.0, limit: int = 100) -> dict:
        deadline = time.monotonic() + max(0.0, timeout)
        with self.cv:
            while self.seq <= cursor:
                remaining = deadline - time.monotonic()
                if remaining <= 0:
                    return {"events": [], "count": 0, "cursor": self.seq, "timeout": True}
                self.cv.wait(timeout=min(remaining, 1.0))
            rows = [e for e in self.events if e["seq"] > cursor][:limit]
            return {"events": rows, "count": len(rows), "cursor": self.seq, "timeout": False}


class AuditLog:
    """Append-only, hash-chained audit log (JSONL)."""

    GENESIS = "0" * 64

```

```
    def __init__(self, path=None, persist: bool = True, max_memory: int = 5000):
        self.path = Path(path) if path else DEFAULT_STATE_DIR / "audit.jsonl"
        self.persist = bool(persist)
        self.lock = threading.Lock()
        self.entries = deque(maxlen=max_memory)
        self.last_hash = self.GENESIS
        self.seq = 0
        self._load()

    def _load(self):
        if not self.persist or not self.path.is_file():
            return
        try:
            with self.path.open("r", encoding="utf-8") as fh:
                for line in fh:
                    line = line.strip()
                    if not line:
                        continue
                    try:
                        row = json.loads(line)
                    except ValueError:
                        continue
                    self.entries.append(row)
                    self.last_hash = row.get("hash") or self.last_hash
                    self.seq = max(self.seq, int(row.get("seq") or 0))
        except OSError:
            pass

    def _read_disk_tail(self, fh) -> dict:
        """Read the last complete JSONL record from an open file handle."""
        try:
            fh.seek(0, os.SEEK_END)
            size = fh.tell()
            window = min(size, 16384)
            fh.seek(size - window)
            chunk = fh.read()
            lines = [ln for ln in chunk.splitlines() if ln.strip()]
            if not lines:
                return {}
            return json.loads(lines[-1])
        except (OSError, ValueError):
            return {}

    def _sync_head_from_disk(self):
        """Adopt a newer on-disk head so concurrent writers keep one chain."""
        try:
            with self.path.open("r", encoding="utf-8") as fh:
                row = self._read_disk_tail(fh)
        except OSError:
            return
        head = row.get("hash")
        if head and head != self.last_hash:
            self.last_hash = head
            self.seq = max(self.seq, int(row.get("seq") or 0))

    def append(self, action: str, actor: str = "system", target: str = "", detail=None, ip: str = "", outcome: str = "ok") -> dict:
        with self.lock:
            if not self.persist:
                self._append_unlocked(action, actor, target, detail, ip, outcome)
                return dict(self.entries[-1])
```

```
            self.path.parent.mkdir(parents=True, exist_ok=True)
            try:
                with self.path.open("a+", encoding="utf-8") as fh:
                    lock = _file_lock(fh)
                    try:
                        self._sync_head_from_disk()
                        entry = self._append_unlocked(action, actor, target, detail, ip, outcome)
                        fh.write(json.dumps(entry, ensure_ascii=False, separators=(",", ":")) + "\n")
                        fh.flush()
                    finally:
                        _file_unlock(fh, lock)
                return dict(entry)
            except OSError:
                entry = self._append_unlocked(action, actor, target, detail, ip, outcome)
                return dict(entry)

    def _append_unlocked(self, action, actor, target, detail, ip, outcome) -> dict:
        self.seq += 1
        entry = {
            "seq": self.seq,
            "ts": _now(),
            "action": action,
            "actor": actor or "system",
            "target": target or "",
            "outcome": outcome,
            "ip": ip or "",
            "detail": detail if isinstance(detail, (dict, list, str, int, float, type(None))) else str(detail),
            "prev": self.last_hash,
        }
        canonical = json.dumps(entry, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
        entry["hash"] = hashlib.sha256(canonical.encode("utf-8")).hexdigest()
        self.last_hash = entry["hash"]
        self.entries.append(entry)
        return entry

    def tail(self, limit: int = 100, action: str = None) -> dict:
        with self.lock:
            rows = list(self.entries)
        if action:
            rows = [r for r in rows if action in str(r.get("action", ""))]
        rows = rows[-max(1, min(int(limit or 100), 2000)):]
        rows.reverse()
        return {"entries": rows, "count": len(rows), "cursor": self.seq}

    def verify(self) -> dict:
        """Recompute the whole chain. Detects edit, reorder, deletion or insertion."""
        with self.lock:
            rows = list(self.entries)
        prev = self.GENESIS
        for index, row in enumerate(rows):
            stored = row.get("hash")
            body = {k: v for k, v in row.items() if k != "hash"}
            canonical = json.dumps(body, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
            expected = hashlib.sha256(canonical.encode("utf-8")).hexdigest()
            if row.get("prev") != prev:
                return {
                    "ok": False,
                    "checked": index + 1,
                    "brokenAt": row.get("seq"),
                    "reason": "prev-hash mismatch (entry inserted, removed or reordered)",
```

```
                }
            if expected != stored:
                return {
                    "ok": False,
                    "checked": index + 1,
                    "brokenAt": row.get("seq"),
                    "reason": "content hash mismatch (entry was edited)",
                }
            prev = stored
        return {
            "ok": True,
            "checked": len(rows),
            "head": prev,
            "entries": len(rows),
            "file": str(self.path) if self.persist else None,
        }


class Telemetry:
    """Convenience container wired into BridgeState."""

    def __init__(self, state_dir=None, persist: bool = True):
        state_dir = Path(state_dir) if state_dir else DEFAULT_STATE_DIR
        self.metrics = Metrics()
        self.events = EventStream()
        self.audit = AuditLog(state_dir / "audit.jsonl", persist=persist)
        self.started = _now()

    @property
    def uptime(self) -> float:
        return _now() - self.started

    def record_job(self, job: dict):
        """Called when a job finishes: counters + latency histogram + event."""
        ok = bool(job.get("ok"))
        labels = {"type": job.get("type") or "unknown", "dm": job.get("dm") or job.get("target_dm") or "auto"}
        self.metrics.inc("bridge_jobs_total", 1, dict(labels, outcome="ok" if ok else "error"))
        duration = job.get("duration_ms")
        if isinstance(duration, (int, float)):
            self.metrics.observe("bridge_job_duration_ms", float(duration), labels)
        self.events.emit(
            "job.finish",
            {
                "id": job.get("id"),
                "type": job.get("type"),
                "status": job.get("status"),
                "durationMs": duration,
                "error": job.get("error"),
            },
            level="info" if ok else "error",
        )

```

roblox-bridge/tools/seed_agent_bus.py

129 lines

Idempotent Agent Bus seeding (online or offline)

```python
#!/usr/bin/env python3
"""
seed_agent_bus.py - Populate the live bridge Agent Bus with the partner work order.

Reads `handoff/bus_seed.json` (agents, tasks, messages) and posts it to a running
bridge over HTTP, or writes it into an offline state directory with `--offline`.

Usage:
    python3 tools/seed_agent_bus.py                      # uses BRIDGE_URL/BRIDGE_TOKEN
    python3 tools/seed_agent_bus.py --url http://127.0.0.1:8077 --token <token>
    python3 tools/seed_agent_bus.py --offline --state-dir ./bridge_state
    python3 tools/seed_agent_bus.py --dry-run

Idempotent: re-running matches agents by name and skips tasks whose title
already exists on the board.
"""

import argparse
import json
import os
import sys
import urllib.error
import urllib.request
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent.parent
DEFAULT_SEED = ROOT / "handoff" / "bus_seed.json"


def http(url: str, token: str, payload: dict, timeout: float = 20.0) -> dict:
    data = json.dumps(payload).encode("utf-8")
    headers = {"Content-Type": "application/json"}
    if token:
        headers["X-Arena-Token"] = token
    req = urllib.request.Request(url.rstrip("/") + "/api/bus", data=data, method="POST", headers=headers)
    try:
        with urllib.request.urlopen(req, timeout=timeout) as resp:
            return json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as exc:
        raise RuntimeError(f"HTTP {exc.code}: {exc.read().decode('utf-8', 'replace')[:300]}") from exc
    except urllib.error.URLError as exc:
        raise RuntimeError(f"Cannot reach bridge at {url}: {exc.reason}") from exc


def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--url", default=os.environ.get("BRIDGE_URL", "http://127.0.0.1:8077"))
    ap.add_argument("--token", default=os.environ.get("BRIDGE_TOKEN") or None)
    ap.add_argument("--seed", default=str(DEFAULT_SEED))
    ap.add_argument("--offline", action="store_true", help="Write directly to a local state dir (no server)")
    ap.add_argument("--state-dir", default=None, help="State dir for --offline (default roblox-bridge/bridge_state)")
    ap.add_argument("--dry-run", action="store_true")
    args = ap.parse_args()

    seed_path = Path(args.seed)
    seed = json.loads(seed_path.read_text(encoding="utf-8"))

    if not args.token:
        token_file = HERE.parent / "bridge.token"
```

```
        if token_file.is_file():
            args.token = token_file.read_text(encoding="utf-8").strip() or None

    if args.offline:
        sys.path.insert(0, str(HERE.parent))
        from bus_core import AgentBus

        bus = AgentBus(Path(args.state_dir) / "bus.json" if args.state_dir else None)
        call = lambda payload: bus.handle(payload)  # noqa: E731
        existing_agents = bus.list_agents()["agents"]
        existing_tasks = bus.list_tasks()["tasks"]
    else:
        call = lambda payload: http(args.url, args.token, payload)  # noqa: E731
        existing_agents = call({"action": "agents"}).get("agents", [])
        existing_tasks = call({"action": "tasks"}).get("tasks", [])

    if args.dry_run:
        print(json.dumps({"agents": seed.get("agents", []), "tasks": seed.get("tasks", []),
                          "messages": seed.get("messages", [])}, indent=2, ensure_ascii=False))
        return 0

    registered = {}
    for agent in seed.get("agents", []):
        row = call({
            "action": "register",
            "name": agent["name"],
            "role": agent.get("role", "agent"),
            "capabilities": agent.get("capabilities", []),
        })
        registered[agent["name"]] = row
        print(f"[agent] {row['name']:<18} {row['id']}  role={row['role']}")

    known_titles = {t.get("title") for t in existing_tasks}
    created = 0
    for task in seed.get("tasks", []):
        if task.get("title") in known_titles:
            print(f"[task ] skip (exists) {task['title']}")
            continue
        row = call({
            "action": "task_create",
            "title": task["title"],
            "detail": task.get("detail", ""),
            "agentId": "seed",
            "priority": task.get("priority", "normal"),
            "tags": task.get("tags", []),
        })
        created += 1
        print(f"[task ] {row['id']}  {row['priority']:<8} {row['title']}")

    posted = 0
    for message in (seed.get("messages", []) if (created or not existing_tasks) else []):
        sender = registered.get(message.get("from", ""), {"id": message.get("from", "seed")})
        row = call({
            "action": "post",
            "agentId": sender["id"],
            "subject": message.get("subject", ""),
            "body": message.get("body", ""),
            "kind": message.get("kind", "note"),
            "priority": message.get("priority", "normal"),
        })
```

```
        posted += 1
        print(f"[msg  ] {row['id']}  {row['subject']}")

    print(f"\nseeded {len(registered)} agents, {created} new tasks, {posted} messages")
    return 0


if __name__ == "__main__":
    sys.exit(main())

```

5. Diffs against the previous release

Unified diffs produced with `git diff 92cde4e280cfcc947f204cf7fbf01e329f9a0282 -- <file>`. Lines prefixed + are added, - removed; each file starts on its own page.

File	Added	Removed	Purpose
`roblox-bridge/server.py`	+1029	-64	HTTP bridge, auth scopes, ops endpoints, dashboard tab 7
`roblox-bridge/mcp_core.py`	+196	-2	agent_bus + bridge_ops tools, 3 resources, client helpers
`roblox-bridge/arena_studio.py`	+164	-1	ops / metrics / audit / tunnel / tokens / bus CLI
`roblox-bridge/setup.py`	+63	-29	Supervised tunnel launch + URL watcher
`roblox-bridge/test_bridge_mcp.py`	+319	-5	v5 test coverage (scopes, bus, reaping, audit, tunnel)
`roblox-bridge/mcp_server.py`	+1	-1	Version banner
`roblox-bridge/ab.ps1`	+6	-1	Version banner + ops examples
Total	**+1778**	**-103**	

The diff sections carry the context lines needed to place each change; nothing outside these files is touched by the release.

roblox-bridge/server.py (diff)

1475 lines

HTTP bridge, auth scopes, ops endpoints, dashboard tab 7

```
diff --git a/roblox-bridge/server.py b/roblox-bridge/server.py
index 9bedc28..773dabd 100644
--- a/roblox-bridge/server.py
+++ b/roblox-bridge/server.py
@@ -7,7 +7,7 @@ Simultaneously serves:
1. Roblox Studio local plugin peers (Edit + Play Server + Play Client bridge)
via fast long-polling (`GET /api/poll`, `POST /api/result`).
2. Native MCP Streamable HTTP endpoint (`POST /mcp`, `GET /mcp`, `DELETE /mcp`)
- implementing MCP 2025-06-18 / 2024-11-05 JSON-RPC 2.0 (68 tools, 5 live
+ implementing MCP 2025-06-18 / 2024-11-05 JSON-RPC 2.0 (70 tools, 8 live
resources, 2 resource templates, 4 workflow prompts).
3. REST API for CLI (`arena_studio.py` / `ab.ps1`) & remote cloud agents
(`GET /api/health`, `GET /api/state`, `GET /api/agent-guide`,
@@ -31,9 +31,20 @@ import time
from urllib.parse import parse_qs, urlparse
import uuid

-from mcp_core import BridgeClient, MCPDispatcher, TOOL_SPECS, run_bridge_self_test
-
-VERSION = "4.0.0"
+from mcp_core import (
+ BridgeClient,
+ MCPDispatcher,
+ TOOL_MAP,
+ TOOL_SPECS,
+ is_tool_call_read_only,
+ run_bridge_self_test,
+)
+from auth_core import ROLE_RANK, TokenStore
+from bus_core import AgentBus
+from telemetry import Telemetry
+from tunnel_manager import TunnelManager
+
+VERSION = "5.0.0"
PEER_ONLINE_WINDOW = 5.0 # play DataModel peers are transient; do not route work to stale peers
HERE = Path(__file__).resolve().parent
SNAPSHOT_DIR = HERE / "snapshots"
@@ -120,10 +131,25 @@ def build_agent_guide(state_snapshot: dict, origin: str = "http://127.0.0.1:8077
# ---------------------------------------------------------------------------

class BridgeState:
- def __init__(self, token: str = "", read_only: bool = False, guard_place_id: int = None):
+ def __init__(
+ self,
+ token: str = "",
+ read_only: bool = False,
+ guard_place_id: int = None,
+ state_dir: str = None,
+ auto_tunnel: bool = False,
+ tunnel_flavor: str = "auto",
+ tunnel_token: str = None,
+ tunnel_name: str = None,
+ public_url: str = None,
+ tunnel_binary: str = None,
+ ):
self.token = token or ""
self.read_only = bool(read_only)
self.guard_place_id = int(guard_place_id) if guard_place_id else None
+ self.started = _now()
+ self.state_dir = Path(state_dir) if state_dir else (HERE / "bridge_state")
```

```
+ self.state_dir.mkdir(parents=True, exist_ok=True)

self.lock = threading.RLock()
self.poll_cv = threading.Condition(self.lock)
@@ -151,6 +177,126 @@ class BridgeState:
)
self.mcp_dispatcher = MCPDispatcher(self.mcp_client)

+ # --- v5 ops platform ------------------------------------------------
+ self.telemetry = Telemetry(self.state_dir)
+ self.bus = AgentBus(self.state_dir / "bus.json")
+ self.tokens = TokenStore(self.state_dir / "tokens.json", primary_token=self.token)
+ self.tunnel = TunnelManager(
+ port=int(os.environ.get("BRIDGE_PORT", 8077)),
+ binary=tunnel_binary,
+ flavor=tunnel_flavor,
+ named_token=tunnel_token,
+ name=tunnel_name,
+ public_url=public_url,
+ state_dir=self.state_dir,
+ auto_start=bool(auto_tunnel),
+ )
+ # Job dedupe + retry bookkeeping
+ self.idempotency = {}
+ self._reaper_stop = threading.Event()
+ self._reaper = threading.Thread(target=self._reap_loop, name="job-reaper", daemon=True)
+ self._reaper.start()
+
+ self.telemetry.events.emit("bridge.start", {
+ "version": VERSION,
+ "readOnly": self.read_only,
+ "tools": len(TOOL_SPECS),
+ "authMode": "scoped" if self.token else "open",
+ })
+ self.telemetry.audit.append(
+ "bridge.start",
+ actor="system",
+ detail={"version": VERSION, "readOnly": self.read_only, "authMode": "scoped" if self.token else "open"},
+ )
+
+ # -- auth ------------------------------------------------------------
+
+ def authenticate(self, supplied: str = None, is_loopback: bool = False, required: str = "read"):
+ """Resolve a raw token into an auth context.
+
+ Returns (context, error). `error` is a dict with `code`/`error` when the
+ request must be rejected. Auth is disabled entirely when no token is set,
+ which preserves the historical zero-config localhost experience.
+ """
+ if not self.token and not self.tokens.records:
+ return {"role": "admin", "id": "open", "label": "auth-disabled"}, None
+ record, rate = self.tokens.authenticate(supplied)
+ if record is None:
+ # No anonymous bypass: a configured token is required for every path
+ # that is not explicitly public (dashboard HTML and /api/health).
+ return None, {"code": 401, "error": "bad token"}
+ if rate and not rate.get("allowed"):
+ return None, {
+ "code": 429,
+ "error": "rate limit exceeded for this token",
```

```
+ "retryAfter": rate.get("retryAfter", 1),
+ }
+ if not TokenStore.role_allows(record.role, required):
+ return None, {
+ "code": 403,
+ "error": f"token role '{record.role}' cannot perform '{required}' actions",
+ }
+ ctx = {
+ "role": record.role,
+ "id": record.id,
+ "label": record.label,
+ "remaining": (rate or {}).get("remaining"),
+ }
+ if record.id != "primary":
+ self.telemetry.metrics.inc("bridge_token_requests_total", 1, {"token": record.id, "role": record.role})
+ return ctx, None
+
+ # -- background reaper ----------------------------------------------------
+
+ def _reap_loop(self):
+ while not self._reaper_stop.wait(2.0):
+ try:
+ self.reap_jobs()
+ except Exception: # noqa: BLE001 - the reaper must never die
+ continue
+
+ def reap_jobs(self) -> dict:
+ """Fail timed-out jobs and re-queue work orphaned by a dead/absent peer."""
+ now = _now()
+ recovered = 0
+ timed_out = 0
+ with self.lock:
+ online = {p["peerId"] for p in self._active_peers_unlocked() if p["online"]}
+ for job in list(self.jobs.values()):
+ if job.get("status") != "running":
+ continue
+ claimed = job.get("claimed") or now
+ timeout = float(job.get("timeout") or 120.0)
+ owner = job.get("claimedBy")
+ orphaned = bool(owner) and owner not in online and (now - claimed) > 20.0
+ if orphaned:
+ job["status"] = "queued"
+ job["claimedBy"] = None
+ job["claimed"] = None
+ job["attempts"] = int(job.get("attempts") or 0) + 1
+ job["note"] = (job.get("note") or "") + f" [requeued: peer {owner} went away]"
+ if job["id"] not in self.pending:
+ self.pending.append(job["id"])
+ recovered += 1
+ self.poll_cv.notify_all()
+ self.telemetry.events.emit("job.requeued_orphan", {"id": job["id"], "type": job["type"], "peer": owner}, level="warn")
+ continue
+ if (now - claimed) > timeout:
+ job["status"] = "error"
+ job["ok"] = False
+ job["error"] = f"timeout after {timeout:.0f}s in {'peer ' + owner if owner else 'queue'}"
+ job["finished"] = now
+ job["duration_ms"] = round((now - claimed) * 1000, 1)
+ timed_out += 1
+ evt = self.job_events.get(job["id"])
```

```
+ if evt:
+ evt.set()
+ self.telemetry.record_job(job)
+ self.telemetry.audit.append("job.timeout", actor="reaper", target=job["id"],
+ detail={"type": job["type"], "peer": owner}, outcome="error")
+ self.telemetry.metrics.inc("bridge_jobs_reaped_total", recovered + timed_out)
+ return {"recovered": recovered, "timedOut": timed_out}
+
# -- peer tracking ------------------------------------------------------

def record_poll(self, qs: dict) -> dict:
@@ -285,22 +431,37 @@ class BridgeState:
with self.poll_cv:
while True:
self.peers[peer_info["peerId"]]["lastPoll"] = _now()
+ best = None
for job_id in list(self.pending):
job = self.jobs.get(job_id)
if not job or job["status"] != "queued":
self.pending.remove(job_id)
continue
- if self._peer_matches_job_unlocked(peer_info, job):
- self.pending.remove(job_id)
- job["status"] = "running"
- job["claimed"] = _now()
- job["claimedBy"] = peer_info["peerId"]
- job["dm"] = peer_info["dm"]
- return {
- "id": job["id"],
- "type": job["type"],
- "payload": job["payload"],
- }
+ if not self._peer_matches_job_unlocked(peer_info, job):
+ continue
+ # Priority first, then FIFO, so a critical agent probe is not
+ # stuck behind a long bulk-build job from another agent.
+ rank = (int(job.get("priority", 5)), job.get("created", 0))
+ if best is None or rank < best[0]:
+ best = (rank, job_id)
+ if best is not None:
+ job_id = best[1]
+ job = self.jobs[job_id]
+ self.pending.remove(job_id)
+ job["status"] = "running"
+ job["claimed"] = _now()
+ job["claimedBy"] = peer_info["peerId"]
+ job["dm"] = peer_info["dm"]
+ job["attempts"] = int(job.get("attempts") or 0) + 1
+ self.telemetry.events.emit("job.claim", {
+ "id": job["id"], "type": job["type"], "peer": peer_info["peerId"],
+ "priority": job.get("priority", 5),
+ })
+ return {
+ "id": job["id"],
+ "type": job["type"],
+ "payload": job["payload"],
+ }

remaining = deadline - time.monotonic()
if remaining <= 0:
```

```
@@ -325,6 +486,10 @@ class BridgeState:
dm: str = None,
peer_id: str = None,
place_id: int = None,
+ idempotency_key: str = None,
+ priority: int = 5,
+ max_retries: int = 0,
+ timeout: float = None,
):
payload_dict = dict(payload or {}) if isinstance(payload, dict) else {}

@@ -351,7 +516,28 @@ class BridgeState:

effective_place = place_id if place_id is not None else self.guard_place_id

+ priority = max(0, min(int(priority if priority is not None else 5), 9))
+ max_retries = max(0, min(int(max_retries or 0), 5))
+ idempotency_key = (idempotency_key or "").strip()[:200] or None
+
with self.poll_cv:
+ now = _now()
+ # Purge idempotency keys older than an hour so the map stays bounded.
+ if len(self.idempotency) > 512:
+ for key in [k for k, v in self.idempotency.items() if now - v.get("ts", 0) > 3600]:
+ self.idempotency.pop(key, None)
+
+ if idempotency_key:
+ prior = self.idempotency.get(idempotency_key)
+ prior_job = self.jobs.get(prior["id"]) if prior else None
+ if prior_job and prior_job.get("status") not in ("error", "cancelled"):
+ prior_job["deduplicated"] = int(prior_job.get("deduplicated") or 0) + 1
+ self.telemetry.metrics.inc("bridge_job_dedupe_total", 1, {"type": kind})
+ self.telemetry.events.emit("job.dedupe", {
+ "id": prior_job["id"], "type": kind, "key": idempotency_key,
+ })
+ return prior_job, self.job_events.setdefault(prior_job["id"], threading.Event())
+
jid = uuid.uuid4().hex[:12]
done_evt = threading.Event()
job = {
@@ -365,7 +551,7 @@ class BridgeState:
"dm": None,
"claimedBy": None,
"status": "queued",
- "created": _now(),
+ "created": now,
"claimed": None,
"finished": None,
"duration_ms": None,
@@ -373,29 +559,82 @@ class BridgeState:
"result": None,
"error": None,
"logs": [],
+ "priority": priority,
+ "maxRetries": max_retries,
+ "retries": 0,
+ "attempts": 0,
+ "timeout": float(timeout) if timeout else None,
+ "idempotencyKey": idempotency_key,
+ "deduplicated": 0,
```

```
}
self.jobs[jid] = job
self.job_events[jid] = done_evt
self.pending.append(jid)
self.history.append(jid)
+ if idempotency_key:
+ self.idempotency[idempotency_key] = {"id": jid, "ts": now}
while len(self.jobs) > 600:
oldest = next(iter(self.jobs))
if oldest == jid:
break
self.jobs.pop(oldest, None)
self.job_events.pop(oldest, None)
+ self.telemetry.metrics.inc("bridge_jobs_enqueued_total", 1, {"type": kind})
+ self.telemetry.events.emit("job.enqueue", {
+ "id": jid, "type": kind, "dm": effective_dm, "priority": priority,
+ "note": note, "idempotencyKey": idempotency_key,
+ })
+ if kind in self.MUTATING_JOB_TYPES:
+ self.telemetry.audit.append(
+ "job.enqueue", actor=note or "agent", target=jid,
+ detail={"type": kind, "dm": effective_dm, "priority": priority},
+ )
self.poll_cv.notify_all()
return job, done_evt

+ TRANSIENT_ERROR_HINTS = (
+ "timeout", "timed out", "time-out", "peer", "disconnect", "connection",
+ "econn", "temporarily", "busy", "try again", "socket", "network",
+ "unknown job type", "not ready",
+ )
+
+ def _is_transient_error(self, body: dict, job: dict) -> bool:
+ if body.get("retryable") is True:
+ return True
+ message = str(body.get("error") or "").lower()
+ if not message:
+ return False
+ return any(hint in message for hint in self.TRANSIENT_ERROR_HINTS)
+
def complete_job(self, body: dict) -> bool:
jid = body.get("id")
- with self.lock:
+ with self.poll_cv:
job = self.jobs.get(jid)
if not job:
return False
now = _now()
- job["status"] = "done" if body.get("ok") else "error"
- job["ok"] = bool(body.get("ok"))
+ ok = bool(body.get("ok"))
+ if not ok and int(job.get("retries") or 0) < int(job.get("maxRetries") or 0) \
+ and self._is_transient_error(body, job):
+ job["retries"] = int(job.get("retries") or 0) + 1
+ job["attempts"] = int(job.get("attempts") or 0)
+ job["status"] = "queued"
+ job["claimedBy"] = None
+ job["claimed"] = None
+ job["error"] = None
+ job["note"] = (job.get("note") or "") + f" [retry {job['retries']}: {str(body.get('error'))[:120]}]"
```

```
+ if jid not in self.pending:
+ self.pending.append(jid)
+ self.telemetry.metrics.inc("bridge_job_retries_total", 1, {"type": job["type"]})
+ self.telemetry.events.emit("job.retry", {
+ "id": jid, "type": job["type"], "retry": job["retries"],
+ "error": str(body.get("error"))[:200],
+ }, level="warn")
+ self.poll_cv.notify_all()
+ return True
+
+ job["status"] = "done" if ok else "error"
+ job["ok"] = ok
job["result"] = body.get("result")
job["error"] = body.get("error")
job["logs"] = body.get("logs") or []
@@ -409,6 +648,12 @@ class BridgeState:
evt = self.job_events.get(jid)
if evt:
evt.set()
+ self.telemetry.record_job(job)
+ if not ok:
+ self.telemetry.audit.append(
+ "job.error", actor=job.get("claimedBy") or "studio", target=jid,
+ detail={"type": job["type"], "error": str(job.get("error"))[:300]}, outcome="error",
+ )
return True

def cancel_job(self, jid: str) -> dict:
@@ -515,6 +760,7 @@ class BridgeState:
with self.lock:
online, st = self.legacy_studio_summary_unlocked()
peers = self._active_peers_unlocked()
+ tunnel_state = self.tunnel.state
return {
"ok": True,
"version": VERSION,
@@ -525,6 +771,15 @@ class BridgeState:
"peers": peers,
"queued": len(self.pending),
"jobs": len(self.jobs),
+ "uptimeSeconds": round(_now() - self.started, 1),
+ "authMode": "scoped" if self.token else "open",
+ "tunnel": {
+ "state": tunnel_state,
+ "publicUrl": self.tunnel.public_url,
+ "localUrl": self.tunnel.local_url,
+ },
+ "agents": self.bus.stats()["agents"],
+ "agentsOnline": self.bus.stats()["agentsOnline"],
}
if op == "state":
with self.lock:
@@ -542,6 +797,12 @@ class BridgeState:
"snapshots": len(self.snapshots),
"recent": recent,
"server_time": _now(),
+ "uptimeSeconds": round(_now() - self.started, 1),
+ "bus": self.bus.stats(),
+ "tunnel": {
+ "state": self.tunnel.state,
```

```
+ "publicUrl": self.tunnel.public_url,
+ },
}
if op == "peers":
with self.lock:
@@ -565,14 +826,113 @@ class BridgeState:
dm=body.get("dm"),
peer_id=body.get("peer_id"),
place_id=body.get("place_id"),
+ idempotency_key=body.get("idempotencyKey") or body.get("idempotency_key"),
+ priority=body.get("priority", 5),
+ max_retries=body.get("maxRetries", body.get("max_retries", 0)),
+ timeout=body.get("timeout"),
)
wait = float(body.get("wait") or 0)
if wait > 0:
evt.wait(timeout=min(wait, 120.0))
with self.lock:
return dict(self.jobs[job["id"]])
+ if op == "bus":
+ return self.bus.handle(body)
+ if op == "tunnel":
+ return self.tunnel_action(body)
+ if op == "tokens":
+ return self.tokens.handle(body)
+ if op == "metrics":
+ return self.metrics_snapshot()
+ if op == "audit":
+ if body.get("verify"):
+ return self.telemetry.audit.verify()
+ return self.telemetry.audit.tail(int(body.get("limit") or 100), body.get("action"))
+ if op == "events":
+ if body.get("wait") is not None:
+ return self.telemetry.events.tail(int(body.get("cursor") or 0), float(body.get("wait") or 0), int(body.get("limit") or 100))
+ return self.telemetry.events.since(int(body.get("cursor") or 0), int(body.get("limit") or 200), body.get("kinds"))
+ if op in ("ops", "ops_report"):
+ return self.ops_report()
+ if op == "queue_clear":
+ cancelled = self.clear_queue()
+ self.telemetry.audit.append("queue.clear", actor="internal", detail={"cancelled": cancelled})
+ return {"cancelled": cancelled}
+ if op == "reap":
+ return self.reap_jobs()
raise ValueError(f"Unknown internal op: {op}")

+ # -- tunnel control ------------------------------------------------------
+
+ def tunnel_action(self, payload: dict) -> dict:
+ """Refuse to publish an unauthenticated bridge, then delegate."""
+ action = ((payload or {}).get("action") or "status").lower()
+ if action in ("start", "restart") and not self.token:
+ status = self.tunnel.status()
+ status.update({
+ "state": "blocked",
+ "running": False,
+ "lastError": (
+ "Refusing to expose the bridge publicly without a token. "
+ "Run `python setup.py` (it writes bridge.token) or start server.py with --token."
+ ),
+ "security": "blocked_no_token",
```

```
+ })
+ self.telemetry.events.emit("tunnel.blocked", {"reason": "no_token"}, level="warn")
+ self.telemetry.audit.append("tunnel.blocked", actor="system", detail={"reason": "no_token"}, outcome="denied")
+ return status
+ return self.tunnel.handle(payload)
+
+ # -- ops reporting ------------------------------------------------------
+
+ def metrics_snapshot(self) -> dict:
+ with self.lock:
+ online, _ = self.legacy_studio_summary_unlocked()
+ peers_online = sum(1 for p in self._active_peers_unlocked() if p["online"])
+ queued = len(self.pending)
+ running = sum(1 for j in self.jobs.values() if j.get("status") == "running")
+ jobs_total = len(self.jobs)
+ online_flag = online
+ bus_stats = self.bus.stats()
+ t = self.telemetry.metrics
+ t.gauge("bridge_peers_online", peers_online)
+ t.gauge("bridge_studio_connected", 1 if online_flag else 0)
+ t.gauge("bridge_queue_depth", queued)
+ t.gauge("bridge_jobs_running", running)
+ t.gauge("bridge_jobs_tracked", jobs_total)
+ t.gauge("bridge_agents_online", bus_stats["agentsOnline"])
+ t.gauge("bridge_tasks_open", sum(
+ v for k, v in bus_stats["taskCounts"].items() if k in ("todo", "in_progress", "blocked")
+ ))
+ t.gauge("bridge_uptime_seconds", round(_now() - self.started, 1))
+ t.gauge("bridge_tunnel_online", 1 if self.tunnel.state == "online" else 0)
+ return t.snapshot(uptime=_now() - self.started)
+
+ def ops_report(self) -> dict:
+ health = self.direct_dispatch("health", {})
+ metrics = self.metrics_snapshot()
+ bus_stats = self.bus.stats()
+ tasks = self.bus.list_tasks(include_done=False)
+ audit = self.telemetry.audit.tail(10)
+ with self.lock:
+ recent = [summary(self.jobs[i]) for i in list(self.history)[-12:]][::-1]
+ return {
+ "ok": True,
+ "version": VERSION,
+ "uptimeSeconds": round(_now() - self.started, 1),
+ "health": health,
+ "tunnel": self.tunnel.status(),
+ "metrics": {
+ "counters": {f"{c['name']}{json.dumps(c['labels'], sort_keys=True) if c['labels'] else ''}": c["value"] for c in metrics["counters"][:80]},
+ "histograms": metrics["histograms"][:20],
+ },
+ "bus": bus_stats,
+ "openTasks": tasks["tasks"][:20],
+ "audit": audit["entries"],
+ "recentJobs": recent,
+ "servedAt": _now(),
+ }
+

def summary(job: dict) -> dict:
out = job.get("result")
@@ -650,17 +1010,98 @@ def make_http_handler(state: BridgeState)::
```

```
except Exception:
return {"_raw": raw.decode("utf-8", "replace")}

- def _authorized(self, qs: dict) -> bool:
- if not state.token:
- return True
+ @staticmethod
+ def _int_qs(qs: dict, key: str, default: int = 0, minimum: int = 0, maximum: int = 10000) -> int:
+ try:
+ value = int((qs.get(key) or [default])[0])
+ except (TypeError, ValueError):
+ return default
+ return max(minimum, min(value, maximum))
+
+ def _supplied_token(self, qs: dict):
bearer = self.headers.get("Authorization", "")
bearer_tok = bearer[7:].strip() if bearer.lower().startswith("bearer ") else None
- supplied = (
+ return (
qs.get("token", [None])[0]
or self.headers.get("X-Arena-Token")
or bearer_tok
)
- return supplied == state.token
+
+ def _auth_ctx(self, qs: dict, required: str = "read", allow_loopback: bool = False):
+ """Return (context, error). Auth is a no-op when no token is configured."""
+ supplied = self._supplied_token(qs)
+ ctx, err = state.authenticate(
+ supplied,
+ is_loopback=self._is_loopback(),
+ required=required,
+ )
+ if err and allow_loopback and required == "read" and not supplied and self._is_loopback():
+ # Backwards-compatible convenience for local dashboards/CLIs on
+ # read-only discovery endpoints. Never applied to /mcp or writes.
+ return {"role": "read", "id": "loopback", "label": "loopback"}, None
+ return ctx, err
+
+ def _authorized(self, qs: dict) -> bool:
+ ctx, err = self._auth_ctx(qs, "read")
+ return err is None
+
+ def _require(self, qs: dict, required: str = "read", allow_loopback: bool = False):
+ """Scope check that writes the error response itself. Returns ctx or None."""
+ ctx, err = self._auth_ctx(qs, required, allow_loopback=allow_loopback)
+ if err:
+ headers = None
+ if err.get("code") == 429:
+ headers = {"Retry-After": str(err.get("retryAfter", 1))}
+ payload = {"error": err.get("error")}
+ if err.get("retryAfter"):
+ payload["retryAfter"] = err["retryAfter"]
+ self._send(err.get("code", 401), payload, extra_headers=headers)
+ state.telemetry.metrics.inc(
+ "bridge_auth_rejections_total", 1,
+ {"code": str(err.get("code")), "path": urlparse(self.path).path},
+ )
+ return None
+ return ctx
```

```
+
+ # -- Server-Sent Events ------------------------------------------------
+
+ def _stream_events(self, qs: dict):
+ cursor = self._int_qs(qs, "since", 0) or self._int_qs(qs, "cursor", 0)
+ kinds = (qs.get("kinds") or [None])[0]
+ try:
+ max_seconds = min(float((qs.get("maxSeconds") or ["300"])[0] or 300), 900)
+ except (TypeError, ValueError):
+ max_seconds = 300.0
+ self.close_connection = True
+ self.send_response(200)
+ self.send_header("Content-Type", "text/event-stream; charset=utf-8")
+ self.send_header("Cache-Control", "no-store")
+ self.send_header("Connection", "close")
+ self.send_header("X-Accel-Buffering", "no")
+ self.send_header("Access-Control-Allow-Origin", "*")
+ self.end_headers()
+
+ def _write(chunk: str):
+ self.wfile.write(chunk.encode("utf-8"))
+ self.wfile.flush()
+
+ started = time.monotonic()
+ _write(": arena bridge event stream\n\n")
+ try:
+ if cursor == 0:
+ seed = state.telemetry.events.since(0, 50, kinds)
+ for event in seed["events"]:
+ _write(f"id: {event['seq']}\ndata: {json.dumps(event)}\n\n")
+ cursor = seed["cursor"]
+ while time.monotonic() - started < max_seconds:
+ batch = state.telemetry.events.tail(cursor, timeout=10.0)
+ for event in batch["events"]:
+ _write(f"id: {event['seq']}\ndata: {json.dumps(event)}\n\n")
+ cursor = event["seq"]
+ if batch.get("timeout"):
+ _write(": keepalive\n\n")
+ except (BrokenPipeError, ConnectionResetError, OSError):
+ return

def do_OPTIONS(self):
self._send(204, b"")
@@ -722,13 +1163,17 @@ def make_http_handler(state: BridgeState):
"bridgeDir": str(HERE) if is_local else "",
"mcpServerPath": str(HERE / "mcp_server.py") if is_local else "roblox-bridge/mcp_server.py",
"cliPath": str(HERE / "arena_studio.py") if is_local else "roblox-bridge/arena_studio.py",
+ "version": VERSION,
+ "toolCount": len(TOOL_SPECS),
+ "authRoles": ["read", "write", "admin"],
"tools": tools_lite,
},
)

if path == "/mcp":
- if not self._authorized(qs):
- return self._send(401, {"error": "unauthorized"})
+ ctx = self._require(qs, "read")
+ if not ctx:
+ return
```

```
return self._send(
200,
{
@@ -737,24 +1182,29 @@ def make_http_handler(state: BridgeState):
"transport": "streamable-http",
"tools": len(TOOL_SPECS),
"readOnly": state.read_only,
+ "role": ctx["role"],
},
extra_headers={"Mcp-Session-Id": state.session_id},
)

+ # Public, unauthenticated liveness probe used by the tunnel supervisor
+ # and uptime monitors. Contains no place data or secrets.
if path == "/api/health":
return self._send(200, state.direct_dispatch("health", {}))

- if path == "/api/self-test":
- if not self._authorized(qs):
- return self._send(401, {"error": "bad token"})
- return self._send(200, run_bridge_self_test(state.mcp_client, include_studio_smoke=True, timeout=180))
-
if path == "/api/state":
+ if not self._require(qs, "read"):
+ return
return self._send(200, state.direct_dispatch("state", {}))

+ if path == "/api/self-test":
+ if not self._require(qs, "read"):
+ return
+ return self._send(200, run_bridge_self_test(state.mcp_client, include_studio_smoke=True, timeout=180))
+
if path == "/api/agent-guide":
- if not self._authorized(qs) and not self._is_loopback():
- return self._send(401, {"error": "bad token"})
+ if not self._require(qs, "read", allow_loopback=True):
+ return
host_hdr = self.headers.get("Host") or "127.0.0.1:8077"
proto = "https" if self.headers.get("CF-Connecting-IP") else "http"
return self._send(
@@ -763,18 +1213,72 @@ def make_http_handler(state: BridgeState):
)

if path == "/api/peers":
- if not self._authorized(qs):
- return self._send(401, {"error": "bad token"})
+ if not self._require(qs, "read"):
+ return
return self._send(200, state.direct_dispatch("peers", {}))

+ # --- v5 observability & collaboration --------------------------------
+ if path == "/api/ops":
+ if not self._require(qs, "read"):
+ return
+ return self._send(200, state.ops_report())
+
+ if path == "/api/metrics":
+ if not self._require(qs, "read"):
+ return
+ fmt = (qs.get("format") or ["json"])[0].lower()
```

```
+ if fmt in ("prometheus", "prom", "text"):
+ body = state.telemetry.metrics.prometheus()
+ return self._send(200, body, "text/plain; version=0.0.4; charset=utf-8")
+ return self._send(200, state.metrics_snapshot())
+
+ if path == "/api/events":
+ if not self._require(qs, "read"):
+ return
+ if (qs.get("once") or qs.get("snapshot") or ["0"])[0] in ("1", "true", "yes"):
+ cursor = self._int_qs(qs, "since", 0) or self._int_qs(qs, "cursor", 0)
+ limit = self._int_qs(qs, "limit", 100, 1, 1000)
+ return self._send(200, state.telemetry.events.since(cursor, limit, (qs.get("kinds") or [None])[0]))
+ return self._stream_events(qs)
+
+ if path == "/api/audit":
+ if not self._require(qs, "read"):
+ return
+ if (qs.get("verify") or ["0"])[0] in ("1", "true", "yes"):
+ return self._send(200, state.telemetry.audit.verify())
+ limit = self._int_qs(qs, "limit", 100, 1, 2000)
+ return self._send(200, state.telemetry.audit.tail(limit, (qs.get("action") or [None])[0]))
+
+ if path == "/api/bus":
+ if not self._require(qs, "read"):
+ return
+ payload = {k: v[0] for k, v in qs.items() if k != "token"}
+ payload.setdefault("action", "stats")
+ try:
+ return self._send(200, state.bus.handle(payload))
+ except KeyError as exc:
+ return self._send(404, {"error": str(exc)})
+ except ValueError as exc:
+ return self._send(400, {"error": str(exc)})
+
+ if path == "/api/tunnel":
+ if not self._require(qs, "read"):
+ return
+ return self._send(200, state.tunnel.status())
+
+ if path == "/api/tokens":
+ if not self._require(qs, "admin"):
+ return
+ return self._send(200, state.tokens.list())
+
if path == "/api/snapshots":
- if not self._authorized(qs):
- return self._send(401, {"error": "bad token"})
+ if not self._require(qs, "read"):
+ return
return self._send(200, state.handle_snapshots({"action": "list"}))

if path.startswith("/api/snapshots/"):
- if not self._authorized(qs):
- return self._send(401, {"error": "bad token"})
+ if not self._require(qs, "read"):
+ return
sid = path.rsplit("/", 1)[-1]
try:
return self._send(200, state.handle_snapshots({"action": "get", "id": sid}))
@@ -783,15 +1287,15 @@ def make_http_handler(state: BridgeState):
```

```
 
# Studio plugin long-polls here
if path == "/api/poll":
- if not self._authorized(qs):
- return self._send(401, {"error": "bad token"})
+ if not self._require(qs, "read"):
+ return
peer_info = state.record_poll(qs)
job = state.poll_for_job(peer_info, wait_seconds=8.0)
return self._send(200, {"job": job})

if path.startswith("/api/jobs/"):
- if not self._authorized(qs) and not self._is_loopback():
- return self._send(401, {"error": "bad token"})
+ if not self._require(qs, "read", allow_loopback=True):
+ return
job_id = path.rsplit("/", 1)[-1]
with state.lock:
job = state.jobs.get(job_id)
@@ -800,8 +1304,8 @@ def make_http_handler(state: BridgeState):
return self._send(200, job)

if path == "/api/jobs":
- if not self._authorized(qs) and not self._is_loopback():
- return self._send(401, {"error": "bad token"})
+ if not self._require(qs, "read", allow_loopback=True):
+ return
with state.lock:
return self._send(200, {"jobs": [summary(state.jobs[i]) for i in state.history]})

@@ -812,11 +1316,12 @@ def make_http_handler(state: BridgeState):
qs = parse_qs(url.query)
path = url.path.rstrip("/") or "/"

- if not self._authorized(qs):
- return self._send(401, {"error": "bad token"})
-
- # Native MCP Streamable HTTP endpoint (JSON-RPC 2.0)
+ # Native MCP Streamable HTTP endpoint (JSON-RPC 2.0). The token role
+ # determines whether mutating tools are advertised/callable.
if path == "/mcp":
+ ctx = self._require(qs, "read")
+ if not ctx:
+ return
raw = self._read_body_raw()
try:
rpc_in = json.loads(raw.decode("utf-8"))
@@ -826,6 +1331,9 @@ def make_http_handler(state: BridgeState):
{"jsonrpc": "2.0", "id": None, "error": {"code": -32700, "message": f"Parse error: {exc}"}},
extra_headers={"Mcp-Session-Id": state.session_id},
)
+ rpc_in, scope_error = self._enforce_mcp_scope(rpc_in, ctx)
+ if scope_error is not None:
+ return self._send(200, scope_error, extra_headers={"Mcp-Session-Id": state.session_id})
rpc_out = state.mcp_dispatcher.handle_payload(rpc_in)
if rpc_out is None:
return self._send(202, b"", "text/plain; charset=utf-8", {"Mcp-Session-Id": state.session_id})
@@ -833,8 +1341,12 @@ def make_http_handler(state: BridgeState):

body = self._body()
```

```
 
- # Enqueue job for Studio
+ # Enqueue job for Studio (requires write scope; admin when read-only
+ # inspector mode is active, since that setting is a safety control).
if path == "/api/jobs":
+ ctx = self._require(qs, "write")
+ if not ctx:
+ return
kind = body.get("type")
if not kind:
return self._send(400, {"error": "type is required"})
@@ -842,10 +1354,14 @@ def make_http_handler(state: BridgeState):
job, evt = state.enqueue_job(
kind=kind,
payload=body.get("payload"),
- note=body.get("note"),
+ note=body.get("note") or f"api:{ctx['label']}",
dm=body.get("dm"),
peer_id=body.get("peer_id"),
place_id=body.get("place_id"),
+ idempotency_key=body.get("idempotencyKey") or self.headers.get("Idempotency-Key"),
+ priority=body.get("priority", 5),
+ max_retries=body.get("maxRetries", 0),
+ timeout=body.get("timeout"),
)
except PermissionError as exc:
return self._send(403, {"error": str(exc)})
@@ -855,9 +1371,16 @@ def make_http_handler(state: BridgeState):
evt.wait(timeout=min(wait, 120.0))
with state.lock:
return self._send(200, state.jobs[job["id"]])
- return self._send(201, {"id": job["id"], "status": job["status"]})
+ return self._send(201, {
+ "id": job["id"],
+ "status": job["status"],
+ "priority": job.get("priority"),
+ "deduplicated": job.get("deduplicated", 0),
+ })
 
if path.startswith("/api/jobs/") and path.endswith("/cancel"):
+ if not self._require(qs, "write"):
+ return
parts = path.split("/")
job_id = parts[3] if len(parts) >= 5 else ""
try:
@@ -866,9 +1389,16 @@ def make_http_handler(state: BridgeState):
return self._send(404, {"error": str(exc)})
 
if path == "/api/queue/clear":
- return self._send(200, {"cancelled": state.clear_queue()})
+ ctx = self._require(qs, "admin")
+ if not ctx:
+ return
+ cancelled = state.clear_queue()
+ state.telemetry.audit.append("queue.clear", actor=ctx["label"], detail={"cancelled": cancelled})
+ return self._send(200, {"cancelled": cancelled})
 
if path == "/api/snapshots":
+ if not self._require(qs, "write"):
+ return
```

```
try:
return self._send(200, state.handle_snapshots(body))
except KeyError as exc:
@@ -876,14 +1406,128 @@ def make_http_handler(state: BridgeState):
except ValueError as exc:
return self._send(400, {"error": str(exc)})

+ # --- v5 collaboration & ops control ---------------------------------
+ if path == "/api/bus":
+ action = (body.get("action") or "stats")
+ required = "write" if action not in (
+ "stats", "agents", "list_agents", "inbox", "thread", "tasks",
+ "list_tasks", "feed", "events", "status",
+ ) else "read"
+ ctx = self._require(qs, required)
+ if not ctx:
+ return
+ try:
+ result = state.bus.handle(body)
+ except KeyError as exc:
+ return self._send(404, {"error": str(exc)})
+ except ValueError as exc:
+ return self._send(400, {"error": str(exc)})
+ state.telemetry.metrics.inc("bridge_bus_actions_total", 1, {"action": action})
+ return self._send(200, result)
+
+ if path == "/api/ops":
+ action = (body.get("action") or "status").lower()
+ required = "read" if action in ("status", "report") else "admin"
+ ctx = self._require(qs, required)
+ if not ctx:
+ return
+ if action in ("status", "report"):
+ return self._send(200, state.ops_report())
+ try:
+ result = state.direct_dispatch(action, body)
+ except ValueError as exc:
+ return self._send(400, {"error": str(exc)})
+ state.telemetry.audit.append(f"ops.{action}", actor=ctx["label"], outcome="ok")
+ return self._send(200, result)
+
+ if path == "/api/tunnel":
+ action = (body.get("action") or "status").lower()
+ required = "admin" if action in ("start", "stop", "restart") else "read"
+ ctx = self._require(qs, required)
+ if not ctx:
+ return
+ try:
+ result = state.tunnel_action(body)
+ except ValueError as exc:
+ return self._send(400, {"error": str(exc)})
+ if action in ("start", "stop", "restart"):
+ state.telemetry.audit.append(f"tunnel.{action}", actor=ctx["label"], outcome="ok")
+ state.telemetry.events.emit("tunnel.action", {"action": action, "actor": ctx["label"]})
+ return self._send(200, result)
+
+ if path == "/api/tokens":
+ ctx = self._require(qs, "admin")
+ if not ctx:
+ return
```

```
+ action = (body.get("action") or "list")
+ try:
+ result = state.tokens.handle(body)
+ except KeyError as exc:
+ return self._send(404, {"error": str(exc)})
+ except ValueError as exc:
+ return self._send(400, {"error": str(exc)})
+ if action in ("create", "revoke"):
+ state.telemetry.audit.append(
+ f"token.{action}", actor=ctx["label"],
+ target=result.get("id") or body.get("id") or "",
+ detail={"role": result.get("role")} if action == "create" else None,
+ )
+ return self._send(200, result)
+
# Studio plugin posts completed job result
if path == "/api/result":
+ if not self._require(qs, "write"):
+ return
if not state.complete_job(body):
return self._send(404, {"error": "no such job"})
return self._send(200, {"ack": True})

return self._send(404, {"error": "not found"})

+ def _enforce_mcp_scope(self, rpc_in, ctx):
+ """Block mutating MCP tool calls for read-scoped tokens (batch aware)."""
+ if ctx.get("role") == "admin" or ctx.get("id") in ("open", "loopback"):
+ return rpc_in, None
+
+ def blocked(item):
+ if not isinstance(item, dict) or item.get("method") != "tools/call":
+ return None
+ params = item.get("params") or {}
+ name = params.get("name")
+ args = params.get("arguments") or {}
+ spec = TOOL_MAP.get(name)
+ if not spec:
+ return None
+ if spec.get("annotations", {}).get("readOnlyHint"):
+ return None
+ if is_tool_call_read_only(name, args):
+ return None
+ message = (
+ f"Blocked by token scope: token '{ctx.get('label')}' has role "
+ f"'{ctx.get('role')}', which cannot call mutating tool '{name}'. "
+ "Mint a write-scoped token via POST /api/tokens or use bridge.token."
+ )
+ return {
+ "jsonrpc": "2.0",
+ "id": item.get("id"),
+ "result": {
+ "isError": True,
+ "content": [{"type": "text", "text": message}],
+ "structuredContent": {"ok": False, "reason": "insufficient_scope", "tool": name},
+ },
+ }
+
+ if isinstance(rpc_in, list):
+ for item in rpc_in:
```

```
+ err = blocked(item)
+ if err is not None:
+ return rpc_in, err
+ return rpc_in, None
+ err = blocked(rpc_in)
+ if err is not None:
+ return rpc_in, err
+ return rpc_in, None
+
return Handler


@@ -968,7 +1612,7 @@ def dashboard_html() -> str:
</style></head><body>

<header>
- <h1>Roblox Studio MCP &amp;amp; Bridge <span class="sub">v4.0</span></h1>
+ <h1>Roblox Studio MCP &amp;amp; Bridge <span class="sub">v5.0</span></h1>
<span id="status" class="pill off">checking&hellip;</span>
<span id="ro-badge" class="pill ro" style="display:none">Read-Only Inspector</span>
<span id="place" class="pill info-pill" style="display:none"></span>
@@ -986,6 +1630,7 @@ def dashboard_html() -> str:
<button class="tab-btn" onclick="showTab('commands', this)">4. Commands &amp;amp; MCP Config</button>
<button class="tab-btn" onclick="showTab('playground', this)">5. Live Tool &amp;amp; Luau Runner</button>
<button class="tab-btn" onclick="showTab('tips', this)">6. Helpful Tips &amp;amp; Gotchas</button>
+ <button class="tab-btn" onclick="showTab('ops', this)">7. Ops Console &amp;amp; Agent Bus</button>
</nav>

<main>
@@ -993,7 +1638,7 @@ def dashboard_html() -> str:
<section id="tab-monitor" class="tab-pane active">
<div class="grid">
<div class="card"><div class="k">Studio Peers</div><div class="v" id="c-peers">0 online</div></div>
- <div class="card"><div class="k">MCP Tools</div><div class="v" id="c-tools">68</div></div>
+ <div class="card"><div class="k">MCP Tools</div><div class="v" id="c-tools">70</div></div>
<div class="card"><div class="k">Queued</div><div class="v" id="c-queued">0</div></div>
<div class="card"><div class="k">Completed</div><div class="v" id="c-done">0</div></div>
<div class="card"><div class="k">Failed</div><div class="v" id="c-fail">0</div></div>
@@ -1251,7 +1896,7 @@ def dashboard_html() -> str:

<div class="prompt-card">
<div class="prompt-head">
- <div><h3><code>.mcp.json</code> / Cursor / Claude Desktop (Full Access)</h3><div class="sub">All 68 tools enabled (3D viewport vision + 3D physics QA + .rbxlx + UI linter + LiveOps + sync)</div></div>
+ <div><h3><code>.mcp.json</code> / Cursor / Claude Desktop (Full Access)</h3><div class="sub">All 70 tools enabled (multi-agent bus + tunnel ops + 3D viewport vision + 3D physics QA + .rbxlx + UI linter + LiveOps + sync)</div></div>
<button class="primary sm" onclick="copyById('cfg-json')">Copy</button>
</div>
<pre id="cfg-json"></pre>
@@ -1297,7 +1942,7 @@ def dashboard_html() -> str:
</div>

<div class="prompt-card">
- <h3>Invoke Any of the 68 MCP Tools Directly</h3>
+ <h3>Invoke Any of the 70 MCP Tools Directly</h3>
<div class="sub" id="tool-desc">Select any MCP tool to inspect its fields and call it over <code>POST /mcp</code>.</div>
<div style="display:flex;gap:8px;align-items:center">
<select id="tool-select" onchange="onSelectTool()"></select>
@@ -1310,6 +1955,100 @@ def dashboard_html() -> str:
</section>
```

```
1079
1080 <!-- ==================== TAB 5: HELPFUL TIPS & GOTCHAS ==================== -->
1081 + <!-- ==================== TAB 7: OPS CONSOLE & AGENT BUS ==================== -->
1082 + <section id="tab-ops" class="tab-pane">
1083 + <div class="grid">
1084 + <div class="card"><div class="k">Tunnel</div><div class="v" id="ops-tunnel-state">stopped</div></div>
1085 + <div class="card"><div class="k">Public URL</div><div class="v" id="ops-tunnel-url" style="font-size:12.5px;word-break:break-all">&mdash;</div></div>
1086 + <div class="card"><div class="k">Agents Online</div><div class="v" id="ops-agents">0</div></div>
1087 + <div class="card"><div class="k">Open Tasks</div><div class="v" id="ops-tasks">0</div></div>
1088 + <div class="card"><div class="k">Queue Depth</div><div class="v" id="ops-queue">0</div></div>
1089 + <div class="card"><div class="k">Audit Entries</div><div class="v" id="ops-audit-count">0</div></div>
1090 + </div>
1091 +
1092 + <div class="two-col">
1093 + <div class="prompt-card">
1094 + <div class="prompt-head">
1095 + <div><h3>Public Tunnel (Supervised)</h3>
1096 + <div class="sub">Auto-restarts, scrapes the public URL, health-probes <code>/api/health</code>, and reconnects after failures. Works with <code>cloudflared</code> or <code>ngrok</code
>.</div>
1097 + </div>
1098 + <span class="pill off" id="ops-tunnel-pill">stopped</span>
1099 + </div>
1100 + <div style="display:flex;gap:6px;flex-wrap:wrap">
1101 + <button class="primary sm" onclick="tunnelAction('start')">Start Tunnel</button>
1102 + <button class="sm" onclick="tunnelAction('restart')">Restart</button>
1103 + <button class="sm danger" onclick="tunnelAction('stop')">Stop</button>
1104 + <button class="sm" onclick="copyText(document.getElementById('ops-tunnel-url').textContent)">Copy URL</button>
1105 + </div>
1106 + <div class="sub" id="ops-tunnel-meta" style="margin-top:6px">Checking tunnel status&hellip;</div>
1107 + <div class="sub" id="ops-tunnel-lan" style="margin-top:4px"></div>
1108 + <pre id="ops-tunnel-log" style="max-height:150px">Tunnel log will appear here.</pre>
1109 + </div>
1110 +
1111 + <div class="prompt-card">
1112 + <div class="prompt-head"><div><h3>Join the Agent Bus</h3>
1113 + <div class="sub">Register this session so other agents (and humans) can see you, message you, and hand work over without collisions.</div></div></div>
1114 + <div style="display:flex;gap:6px;flex-wrap:wrap">
1115 + <input id="bus-name" placeholder="Agent name (e.g. Gemini 4 Argon)" style="flex:1;min-width:190px">
1116 + <input id="bus-role" placeholder="role (builder / qa / reviewer)" style="width:190px">
1117 + <button class="primary sm" onclick="busRegister()">Register</button>
1118 + </div>
1119 + <div style="display:flex;gap:6px;flex-wrap:wrap;margin-top:6px">
1120 + <input id="bus-to" placeholder="to (blank = broadcast)" style="width:180px">
1121 + <input id="bus-subject" placeholder="subject" style="width:180px">
1122 + </div>
1123 + <textarea id="bus-body" placeholder="Message body&hellip; use this to coordinate edits, hand off work, or report a blocker." style="min-height:64px"></textarea>
1124 + <div style="display:flex;gap:6px">
1125 + <button class="sm primary" onclick="busPost()">Post Message</button>
1126 + <button class="sm" onclick="refreshBus()">Refresh Bus</button>
1127 + <button class="sm" onclick="busExport()">Export Handoff</button>
1128 + </div>
1129 + <pre id="bus-out" style="max-height:110px">Bus output appears here.</pre>
1130 + </div>
1131 + </div>
1132 +
1133 + <div class="two-col">
1134 + <div class="prompt-card">
1135 + <h3>Roster</h3>
1136 + <div id="bus-agents" class="sub">No agents registered yet.</div>
1137 + </div>
```

```
+ <div class="prompt-card">
+ <h3>Task Board</h3>
+ <div style="display:flex;gap:6px;margin-bottom:8px">
+ <input id="task-title" placeholder="New task title..." style="flex:1">
+ <select id="task-priority" style="width:110px">
+ <option value="normal">normal</option>
+ <option value="high">high</option>
+ <option value="critical">critical</option>
+ <option value="low">low</option>
+ </select>
+ <button class="primary sm" onclick="createTask()">Add</button>
+ </div>
+ <div id="bus-tasks" style="display:flex;flex-direction:column;gap:6px;max-height:280px;overflow:auto"></div>
+ </div>
+ </div>
+
+ <div class="two-col">
+ <div class="prompt-card">
+ <div class="prompt-head"><h3>Live Event Stream</h3>
+ <button class="sm" onclick="toggleEventStream()" id="ev-btn">Connect</button></div>
+ <div class="sub">Server-Sent Events from every agent, job, tunnel action and audit write.</div>
+ <pre id="ev-feed" style="max-height:240px;overflow:auto">Not connected.</pre>
+ </div>
+ <div class="prompt-card">
+ <div class="prompt-head"><h3>Metrics &amp;amp; Audit</h3>
+ <div style="display:flex;gap:6px">
+ <button class="sm" onclick="refreshMetrics()">Metrics</button>
+ <button class="sm" onclick="verifyAudit()">Verify Chain</button>
+ </div>
+ </div>
+ <pre id="ops-metrics" style="max-height:180px;overflow:auto">Metrics not loaded.</pre>
+ <div class="sub" id="ops-audit-note"></div>
+ <pre id="ops-audit" style="max-height:150px;overflow:auto">Audit log not loaded.</pre>
+ </div>
+ </div>
+ </section>
+
<section id="tab-tips" class="tab-pane">
<div class="two-col" id="tips-grid"></div>
</section>
@@ -1586,7 +2325,7 @@ Start by running a quick recon (\`.\\ab.ps1 state\` and \`.\\ab.ps1 survey\`) so
`Connect directly to my live Roblox Studio Bridge & MCP Server:
- Base URL: ${pubUrl}
- Auth Header: \`Authorization: Bearer ${tok}\` (or \`X-Arena-Token: ${tok}\`)
-- Native MCP Streamable HTTP Endpoint: \`POST ${pubUrl}/mcp\` (68 tools, 16 skills, multimodal 3D viewport vision, 3D physics auditor, runtime mesh capability probe, .rbxlx place export, 14 LiveOps/Monetization systems, 520 3D models, 3,705 raster images + 50 LDR skies (300 cube faces), 45 CC0 WAVs, 4,811 cloud icons)
+- Native MCP Streamable HTTP Endpoint: \`POST ${pubUrl}/mcp\` (70 tools, 16 skills, multimodal 3D viewport vision, 3D physics auditor, runtime mesh capability probe, .rbxlx place export, 14 LiveOps/Monetization systems, 520 3D models, 3,705 raster images + 50 LDR skies (300 cube faces), 45 CC0 WAVs, 4,811 cloud icons)
- Agent Onboarding & Live State Endpoint: \`GET ${pubUrl}/api/agent-guide\`
- Built-in Repo Kit: call \`browse_kit\` for 16 skills, 19 modules, 6 templates, 20 prefabs, 3,755 image/sky-face files, 50 LDR sky panoramas, 520 models, and 45 CC0 audio files. Use \`search_assets\` for paginated type/category/source/license/provenance filters; exact Git blob matches and pack-level attestations are distinguished. Send supported images/audio/models into Studio with \`import_image\`, \`import_audio\`, and \`import_rbxmx\`.

@@ -2097,7 +2836,7 @@ async function initBoot(){

async function tick(){
try {
- const r = await fetch('/api/state', { cache:'no-store' });
+ const r = await fetch('/api/state', { cache:'no-store', headers: authHeaders() });
const s = await r.json();
```

```
const prevPlace = LAST_STATE.studio && LAST_STATE.studio.place;
LAST_STATE = s;
@@ -2161,7 +2900,186 @@ async function tick(){
} catch(_){}
}

-initBoot().then(() => { tick(); setInterval(tick, 1500); });
+// ---------------------------- v5 Ops Console ----------------------------
+let EV_SOURCE = null;
+async function opsJson(path, opts){
+ const r = await fetch(path, Object.assign({cache:'no-store', headers:authHeaders()}, opts || {}));
+ const data = await r.json().catch(() => ({}));
+ if(!r.ok) throw new Error(data.error || ('HTTP ' + r.status));
+ return data;
+}
+async function refreshOps(){
+ try {
+ const ops = await opsJson('/api/ops');
+ const t = ops.tunnel || {};
+ document.getElementById('ops-tunnel-state').textContent = t.state || 'unknown';
+ document.getElementById('ops-tunnel-url').textContent = t.publicUrl || '-';
+ const pill = document.getElementById('ops-tunnel-pill');
+ pill.textContent = t.state || 'unknown';
+ pill.className = 'pill ' + (t.state === 'online' ? 'on' : (t.state === 'degraded' ? 'ro' : 'off'));
+ document.getElementById('ops-agents').textContent = (ops.bus && ops.bus.agentsOnline) || 0;
+ document.getElementById('ops-tasks').textContent = (ops.openTasks || []).length;
+ document.getElementById('ops-queue').textContent = (ops.health && ops.health.queued) || 0;
+ document.getElementById('ops-audit-count').textContent = (ops.audit || []).length;
+ const health = t.health || {};
+ document.getElementById('ops-tunnel-meta').textContent =
+ `flavor=${t.flavor || '-'} · restarts=${t.restarts || 0} · health=${health.ok === null || health.ok === undefined ? 'n/a' : (health.ok ? 'ok' : 'failing')}` +
+ (health.latencyMs != null ? ` · ${health.latencyMs}ms` : '') + (t.lastError ? ' · ' + t.lastError : '');
+ document.getElementById('ops-tunnel-lan').textContent = (t.lanUrls || []).length ? 'LAN: ' + t.lanUrls.join(' · ') : '';
+ document.getElementById('ops-tunnel-log').textContent = (t.logTail || []).join('\n') || 'No tunnel output yet.';
+ if((ops.audit || []).length){
+ document.getElementById('ops-audit').textContent = ops.audit.slice(0,8).map(e => `${e.seq} ${e.action} · ${e.actor} · ${e.outcome}`).join('\n');
+ }
+ renderBus(ops.openTasks || [], null);
+ } catch(_){}
+}
+async function tunnelAction(action){
+ try {
+ const r = await fetch('/api/tunnel', {method:'POST', headers:authHeaders(), body:JSON.stringify({action})});
+ const data = await r.json();
+ if(!r.ok) throw new Error(data.error || ('HTTP ' + r.status));
+ toast('Tunnel ' + action + ' -> ' + (data.state || 'ok'));
+ refreshOps();
+ } catch(err){ toast('Tunnel ' + action + ' failed: ' + err.message); }
+}
+async function busRegister(){
+ const name = document.getElementById('bus-name').value.trim();
+ const role = document.getElementById('bus-role').value.trim() || 'agent';
+ if(!name) return toast('Enter an agent name.');
+ try {
+ const reg = await opsJson('/api/bus', {method:'POST', body:JSON.stringify({action:'register', name, role})});
+ localStorage.setItem('arena_agent_id', reg.id);
+ localStorage.setItem('arena_agent_name', reg.name);
+ document.getElementById('bus-out').textContent = `Registered ${reg.name} as ${reg.id} (${reg.role})`;
+ refreshBus();
+ } catch(err){ toast(err.message); }
```

```
+}
+function currentAgent(){ return localStorage.getItem('arena_agent_id') || 'web-operator'; }
+async function busPost(){
+ const to = document.getElementById('bus-to').value.trim() || null;
+ const subject = document.getElementById('bus-subject').value.trim();
+ const body = document.getElementById('bus-body').value.trim();
+ if(!subject && !body) return toast('Write a subject or body first.');
+ try {
+ const msg = await opsJson('/api/bus', {method:'POST', body:JSON.stringify({
+ action:'post', agentId: currentAgent(), to, subject, body: body || subject, kind:'note', priority:'normal'
+ })});
+ document.getElementById('bus-out').textContent = 'Posted ' + msg.id;
+ document.getElementById('bus-body').value = '';
+ refreshBus();
+ } catch(err){ toast(err.message); }
+}
+async function busExport(){
+ try {
+ const r = await opsJson('/api/bus', {method:'POST', body:JSON.stringify({action:'export', agentId:currentAgent()})});
+ const note = document.getElementById('bus-out');
+ note.textContent = r.markdown;
+ copyText(r.markdown);
+ toast('Handoff brief copied to clipboard.');
+ } catch(err){ toast(err.message); }
+}
+function renderBus(tasks, agents){
+ const taskBox = document.getElementById('bus-tasks');
+ if(!tasks.length){
+ taskBox.innerHTML = '<div class="sub">No open tasks. Create one from any agent with <code>agent_bus {action:"task_create"}</code>.</div>';
+ } else {
+ taskBox.innerHTML = tasks.map(t => {
+ const lease = t.leaseRemainingSeconds != null ? Math.max(0, Math.round(t.leaseRemainingSeconds)) + 's lease' : 'unclaimed';
+ return `<div class="cmd-row" style="align-items:flex-start">
+ <div style="flex:1">
+ <div><strong>${esc(t.title)}</strong> <span class="sub">· ${esc(t.priority)} · ${esc(t.status)} · ${lease}</span></div>
+ <div class="sub">${esc(t.assignee || 'unassigned')} ${t.blockedBy && t.blockedBy.length ? '· blocked by ' + esc(t.blockedBy.join(', ')) : ''}</div>
+ </div>
+ <button class="sm" onclick="claimTask('${esc(t.id)}')">Claim</button>
+ <button class="sm" onclick="completeTask('${esc(t.id)}')">Done</button>
+ </div>`;
+ }).join('');
+ }
+ if(agents){
+ const box = document.getElementById('bus-agents');
+ box.innerHTML = agents.length ? agents.map(a =>
+ `<div style="display:flex;justify-content:space-between;gap:8px;padding:3px 0">
+ <span><strong>${esc(a.name)}</strong> <span class="sub">${esc(a.role)}</span></span>
+ <span class="sub">${esc(a.status)} · ${a.openTasks} tasks · ${a.inboxUnread} unread</span>
+ </div>`).join('') : 'No agents registered yet.';
+ }
+}
+async function refreshBus(){
+ try {
+ const agents = await opsJson('/api/bus?action=agents');
+ const tasks = await opsJson('/api/bus?action=tasks&includeDone=0');
+ renderBus(tasks.tasks || [], agents.agents || []);
+ document.getElementById('ops-agents').textContent = agents.onlineCount || 0;
+ document.getElementById('ops-tasks').textContent = (tasks.tasks || []).length;
+ } catch(_){}
+}
```

```
+async function createTask(){
+ const title = document.getElementById('task-title').value.trim();
+ if(!title) return toast('Enter a task title first.');
+ try {
+ await opsJson('/api/bus', {method:'POST', body:JSON.stringify({
+ action:'task_create', title, priority:document.getElementById('task-priority').value,
+ agentId: currentAgent()
+ })});
+ document.getElementById('task-title').value = '';
+ toast('Task added to the board.');
+ refreshBus();
+ } catch(err){ toast(err.message); }
+}
+async function claimTask(id){
+ try {
+ await opsJson('/api/bus', {method:'POST', body:JSON.stringify({action:'task_claim', taskId:id, agentId:currentAgent()})});
+ toast('Task claimed.');
+ refreshBus();
+ } catch(err){ toast(err.message); }
+}
+async function completeTask(id){
+ try {
+ await opsJson('/api/bus', {method:'POST', body:JSON.stringify({action:'task_complete', taskId:id, agentId:currentAgent(), result:'Completed from the Ops Console'})});
+ toast('Task completed.');
+ refreshBus();
+ } catch(err){ toast(err.message); }
+}
+async function refreshMetrics(){
+ try {
+ const m = await opsJson('/api/metrics');
+ const counters = (m.counters || []).map(c => `${c.name} = ${c.value}${Object.keys(c.labels||{}).length ? ' ' + JSON.stringify(c.labels) : ''}`);
+ const hists = (m.histograms || []).map(h => `${h.name} p50=${h.p50} p95=${h.p95} n=${h.count}`);
+ document.getElementById('ops-metrics').textContent = 'uptime ' + m.uptimeSeconds + 's\n' + counters.concat(hists).join('\n');
+ } catch(err){ document.getElementById('ops-metrics').textContent = err.message; }
+}
+async function verifyAudit(){
+ try {
+ const v = await opsJson('/api/audit?verify=1');
+ document.getElementById('ops-audit-note').textContent = v.ok
+ ? `Chain intact: ${v.checked} entries, head ${String(v.head).slice(0,12)}...`
+ : `CHAIN BROKEN at #${v.brokenAt}: ${v.reason}`;
+ const a = await opsJson('/api/audit?limit=8');
+ document.getElementById('ops-audit').textContent = (a.entries || []).map(e => `${e.seq} ${e.action} · ${e.actor} · ${e.outcome}`).join('\n');
+ } catch(err){ document.getElementById('ops-audit-note').textContent = err.message; }
+}
+function toggleEventStream(){
+ const feed = document.getElementById('ev-feed');
+ const btn = document.getElementById('ev-btn');
+ if(EV_SOURCE){ EV_SOURCE.close(); EV_SOURCE = null; btn.textContent = 'Connect'; feed.textContent += '\n[disconnected]'; return; }
+ const token = getToken();
+ EV_SOURCE = new EventSource('/api/events' + (token ? '?token=' + encodeURIComponent(token) : ''));
+ btn.textContent = 'Disconnect';
+ feed.textContent = '';
+ EV_SOURCE.onmessage = ev => {
+ try {
+ const e = JSON.parse(ev.data);
+ feed.textContent += `${new Date((e.ts||0)*1000).toLocaleTimeString()} ${e.kind} ${JSON.stringify(e.data)}\n`;
+ feed.scrollTop = feed.scrollHeight;
+ } catch(_){}
+ };
```

```
+ EV_SOURCE.onerror = () => { feed.textContent += '\n[stream error - reconnect from the button above]\n'; };
+}
+
+initBoot().then(() => { tick(); refreshOps(); refreshBus(); setInterval(tick, 1500); setInterval(() => {
+ const active = document.getElementById('tab-ops').classList.contains('active');
+ if(active){ refreshOps(); refreshBus(); }
+}, 4000); });
</script></body></html>"""


@@ -2189,16 +3107,35 @@ def create_http_server(
token: str = None,
read_only: bool = False,
guard_place_id: int = None,
+ state_dir: str = None,
+ auto_tunnel: bool = False,
+ tunnel_flavor: str = "auto",
+ tunnel_token: str = None,
+ tunnel_name: str = None,
+ public_url: str = None,
+ tunnel_binary: str = None,
):
resolved_token = load_token_default(token)
state = BridgeState(
token=resolved_token,
read_only=read_only,
guard_place_id=guard_place_id,
+ state_dir=state_dir,
+ auto_tunnel=False, # started after the socket is bound, so the port is known
+ tunnel_flavor=tunnel_flavor,
+ tunnel_token=tunnel_token,
+ tunnel_name=tunnel_name,
+ public_url=public_url,
+ tunnel_binary=tunnel_binary,
)
httpd = ThreadingHTTPServer((host, port), make_http_handler(state))
httpd.daemon_threads = True
httpd.bridge_state = state
+ bound_port = httpd.server_address[1]
+ state.tunnel.port = int(bound_port)
+ state.tunnel.local_url = f"http://127.0.0.1:{bound_port}"
+ if auto_tunnel:
+ state.tunnel_action({"action": "start"})
return httpd, state


@@ -2219,6 +3156,18 @@ def main():
default=int(os.environ["BRIDGE_PLACE_ID"]) if os.environ.get("BRIDGE_PLACE_ID") else None,
help="Optional PlaceId write guard",
)
+ ap.add_argument("--tunnel", action="store_true",
+ help="Supervise a public cloudflared/ngrok tunnel for this bridge (auto-restart, health probes)")
+ ap.add_argument("--tunnel-flavor", default=os.environ.get("BRIDGE_TUNNEL_FLAVOR", "auto"),
+ choices=["auto", "cloudflared", "ngrok"], help="Tunnel provider")
+ ap.add_argument("--tunnel-token", default=os.environ.get("TUNNEL_TOKEN"),
+ help="Cloudflare named-tunnel token (https://one.dash.cloudflare.com)")
+ ap.add_argument("--tunnel-name", default=os.environ.get("BRIDGE_TUNNEL_NAME"),
+ help="Optional hostname label for a named tunnel")
+ ap.add_argument("--public-url", default=os.environ.get("BRIDGE_PUBLIC_URL"),
+ help="Report a tunnel URL you run yourself (disables supervision)")
```

```
+ ap.add_argument("--state-dir", default=os.environ.get("BRIDGE_STATE_DIR"),
+ help="Directory for bus/token/audit/tunnel state (default: roblox-bridge/bridge_state)")
args = ap.parse_args()

srv, state = create_http_server(
@@ -2227,17 +3176,33 @@ def main():
token=args.token,
read_only=args.read_only,
guard_place_id=args.place_id,
+ state_dir=args.state_dir,
+ auto_tunnel=args.tunnel,
+ tunnel_flavor=args.tunnel_flavor,
+ tunnel_token=args.tunnel_token,
+ tunnel_name=args.tunnel_name,
+ public_url=args.public_url,
)
ro_str = " [READ-ONLY INSPECTOR]" if state.read_only else ""
+ bound_port = srv.server_address[1]
print(
- f"[bridge] v{VERSION}{ro_str} listening on {args.host}:{args.port} "
- f"(token {'set' if state.token else 'disabled'}, Control Center http://127.0.0.1:{args.port}/, MCP http://127.0.0.1:{args.port}/mcp)",
+ f"[bridge] v{VERSION}{ro_str} listening on {args.host}:{bound_port} "
+ f"(token {'set' if state.token else 'disabled'}, Control Center http://127.0.0.1:{bound_port}/, "
+ f"MCP http://127.0.0.1:{bound_port}/mcp, ops http://127.0.0.1:{bound_port}/#ops)",
flush=True,
)
+ if args.tunnel:
+ print("[bridge] tunnel supervisor starting (cloudflared/ngrok)...", flush=True)
try:
srv.serve_forever()
except KeyboardInterrupt:
- srv.shutdown()
+ pass
+ finally:
+ try:
+ state._reaper_stop.set()
+ state.tunnel.stop()
+ finally:
+ srv.shutdown()


if __name__ == "__main__":
```

roblox-bridge/mcp_core.py (diff)

258 lines

agent_bus + bridge_ops tools, 3 resources, client helpers

```
diff --git a/roblox-bridge/mcp_core.py b/roblox-bridge/mcp_core.py
index f5fbba3..d7aea04 100644
--- a/roblox-bridge/mcp_core.py
+++ b/roblox-bridge/mcp_core.py
@@ -1,7 +1,7 @@
#!/usr/bin/env python3
"""
mcp_core.py - Zero-dependency Model Context Protocol (MCP) engine & Roblox Studio
-tool suite for Arena <-> Roblox Studio (v4.0.0).
+tool suite for Arena <-> Roblox Studio (v5.0.0).

Supports:
- MCP specification versions 2025-06-18, 2025-03-26, and 2024-11-05
@@ -28,7 +28,7 @@ import urllib.request
from pathlib import Path

MCP_SERVER_NAME = "arena-roblox-studio-mcp"
-MCP_SERVER_VERSION = "4.0.0"
+MCP_SERVER_VERSION = "5.0.0"
SUPPORTED_PROTOCOL_VERSIONS = (
"2025-06-18",
"2025-03-26",
@@ -459,6 +459,72 @@ class BridgeClient:
return self.direct_dispatch("peers", {})
return self._http_request("GET", "/api/peers", timeout=10.0)

+ # -- v5 coordination & ops helpers ----------------------------------------
+
+ def bus_action(self, payload: dict = None) -> dict:
+ payload = dict(payload or {})
+ if self.direct_dispatch:
+ return self.direct_dispatch("bus", payload)
+ return self._http_request("POST", "/api/bus", payload, timeout=30.0)
+
+ def ops_action(self, payload: dict = None) -> dict:
+ p = dict(payload or {})
+ action = (p.get("action") or "status").lower()
+ if self.direct_dispatch:
+ mapping = {
+ "status": ("ops", {}),
+ "metrics": ("metrics", {}),
+ "audit": ("audit", {"limit": p.get("limit", 50)}),
+ "audit_verify": ("audit", {"verify": True}),
+ "events": ("events", {"cursor": p.get("cursor", 0), "limit": p.get("limit", 50)}),
+ "tunnel": ("tunnel", {"action": "status"}),
+ "tunnel_start": ("tunnel", {"action": "start"}),
+ "tunnel_stop": ("tunnel", {"action": "stop"}),
+ "tunnel_restart": ("tunnel", {"action": "restart"}),
+ "queue_clear": ("queue_clear", {}),
+ "reap": ("reap", {}),
+ }
+ if action not in mapping:
+ raise ValueError(f"Unknown bridge_ops action: {action}")
+ op, body = mapping[action]
+ return self.direct_dispatch(op, body)
+
+ if action == "status":
+ return self._http_request("GET", "/api/ops", timeout=20.0)
+ if action == "metrics":
+ fmt = p.get("format") or "json"
```

```
+ if fmt == "prometheus":
+ return {"format": "prometheus", "text": self._http_request_text("/api/metrics?format=prometheus")}
+ return self._http_request("GET", "/api/metrics", timeout=20.0)
+ if action == "audit":
+ return self._http_request("GET", f"/api/audit?limit={int(p.get('limit') or 50)}", timeout=20.0)
+ if action == "audit_verify":
+ return self._http_request("GET", "/api/audit?verify=1", timeout=20.0)
+ if action == "events":
+ return self._http_request(
+ "GET", f"/api/events?once=1&since={int(p.get('cursor') or 0)}&limit={int(p.get('limit') or 50)}", timeout=20.0
+ )
+ if action in ("tunnel", "tunnel_start", "tunnel_stop", "tunnel_restart"):
+ sub = {"tunnel": "status", "tunnel_start": "start", "tunnel_stop": "stop", "tunnel_restart": "restart"}[action]
+ if sub == "status":
+ return self._http_request("GET", "/api/tunnel", timeout=15.0)
+ return self._http_request("POST", "/api/tunnel", {"action": sub}, timeout=90.0)
+ if action in ("queue_clear", "reap"):
+ return self._http_request("POST", "/api/ops", {"action": action}, timeout=30.0)
+ raise ValueError(f"Unknown bridge_ops action: {action}")
+
+ def _http_request_text(self, path: str, timeout: float = 20.0) -> str:
+ url = f"{self.base_url}{path}"
+ headers = {"X-Arena-Token": self.token} if self.token else {}
+ req = urllib.request.Request(url, method="GET", headers=headers)
+ try:
+ with urllib.request.urlopen(req, timeout=timeout) as resp:
+ return resp.read().decode("utf-8", "replace")
+ except urllib.error.HTTPError as exc:
+ return exc.read().decode("utf-8", "replace")[:2000]
+ except urllib.error.URLError as exc:
+ raise BridgeTransportError(f"Cannot reach bridge at {self.base_url} ({exc.reason})") from exc
+
def get_job(self, job_id: str) -> dict:
if self.direct_dispatch:
return self.direct_dispatch("get_job", {"id": job_id})
@@ -1678,6 +1744,88 @@ TOOL_SPECS = [
},
"annotations": _ann("Roblox Open Cloud Publish / DataStore / Messaging", read_only=False, destructive=False, idempotent=False),
},
+
+ # --- Group Z: v5 multi-agent collaboration & bridge operations ----------
+ {
+ "name": "agent_bus",
+ "description": (
+ "Shared multi-agent coordination bus for this bridge (does not touch the Roblox place). "
+ "Register presence, exchange messages, run a claim-based task board with leases, read the activity feed, "
+ "and export a markdown handoff brief. Use it when two or more AI agents (or a human and an agent) share "
+ "one Studio session: `action='register'` first, then `task_claim` before working, `task_complete` with a "
+ "result, and `post` to report progress. Read actions: stats, agents, inbox, tasks, thread, feed. "
+ "Writes: register, heartbeat, post, mark_read, task_create, task_claim, task_renew, task_release, "
+ "task_update, task_complete, task_delete, status."
+ ),
+ "inputSchema": {
+ "type": "object",
+ "properties": {
+ "action": {
+ "type": "string",
+ "enum": [
+ "register", "heartbeat", "status", "agents", "post", "inbox", "thread", "mark_read",
+ "tasks", "task_create", "task_claim", "task_renew", "task_release", "task_update",
```

```
+ "task_complete", "task_delete", "feed", "export", "stats",
+ ],
+ "description": "Bus operation to perform.",
+ },
+ "agentId": {"type": "string", "description": "Your agent id (returned by register) or your registered name."},
+ "createdBy": {"type": "string", "description": "Alias for agentId on `task_create`."},
+ "actor": {"type": "string", "description": "Alias for agentId on `task_update`."},
+ "name": {"type": "string", "description": "Agent display name for `register`."},
+ "role": {"type": "string", "description": "Free-form role label, e.g. 'builder', 'qa', 'reviewer', 'human'."},
+ "capabilities": {"type": "array", "items": {"type": "string"}, "description": "Capability tags for `register`."},
+ "status": {"type": "string", "enum": ["online", "busy", "idle", "offline"], "description": "Presence status for `heartbeat`/`status`."},
+ "to": {"type": "string", "description": "Recipient agent id/name, or omit for a broadcast."},
+ "subject": {"type": "string", "description": "Short message subject."},
+ "body": {"type": "string", "description": "Message body for `post`."},
+ "kind": {"type": "string", "enum": ["note", "question", "answer", "handoff", "alert", "result"], "default": "note"},
+ "priority": {"type": "string", "enum": ["low", "normal", "high", "critical"], "default": "normal"},
+ "threadId": {"type": "string", "description": "Optional thread id to group a conversation."},
+ "since": {"type": "integer", "description": "Cursor for `inbox`/`feed` (use the `cursor` returned last time)."},
+ "limit": {"type": "integer", "description": "Max rows to return (default 50)."},
+ "unreadOnly": {"type": "boolean", "description": "Only unread messages for `inbox`."},
+ "messageIds": {"type": "array", "items": {"type": "string"}, "description": "Messages to mark read."},
+ "all": {"type": "boolean", "description": "Mark every message read."},
+ "taskId": {"type": "string", "description": "Task id for claim/renew/release/update/complete/delete."},
+ "title": {"type": "string", "description": "Task title for `task_create`."},
+ "detail": {"type": "string", "description": "Task detail / acceptance criteria."},
+ "assignee": {"type": "string", "description": "Optional pre-assigned agent id for `task_create`/`task_update`."},
+ "tags": {"type": "array", "items": {"type": "string"}, "description": "Task tags."},
+ "dependsOn": {"type": "array", "items": {"type": "string"}, "description": "Task ids that must complete before this task can be claimed."},
+ "leaseSeconds": {"type": "integer", "description": "Lease duration for `task_claim`/`task_renew` (default 900)."},
+ "result": {"type": "string", "description": "Outcome summary for `task_complete`."},
+ "artifacts": {"type": "array", "items": {"type": "string"}, "description": "Files/URLs produced by the task."},
+ "path": {"type": "string", "description": "Optional file path for `export` (markdown handoff brief)."},
+ "includeDone": {"type": "boolean", "description": "Include finished tasks in `tasks` / `export`."},
+ },
+ },
+ "annotations": _ann("Multi-Agent Coordination Bus", read_only=True, destructive=False, idempotent=False),
+ },
+ {
+ "name": "bridge_ops",
+ "description": (
+ "Operate and observe the bridge itself: consolidated ops report (peers, queue, tunnel, bus, audit), "
+ "Prometheus-style metrics, tamper-evident audit log verification, live event feed, and supervised "
+ "public-tunnel control (start/stop/restart). Read actions: status, metrics, audit, audit_verify, events, "
+ "tunnel. Control: tunnel_start, tunnel_stop, tunnel_restart, queue_clear, reap."
+ ),
+ "inputSchema": {
+ "type": "object",
+ "properties": {
+ "action": {
+ "type": "string",
+ "enum": ["status", "metrics", "audit", "audit_verify", "events", "tunnel", "tunnel_start",
+ "tunnel_stop", "tunnel_restart", "queue_clear", "reap"],
+ "default": "status",
+ },
+ "limit": {"type": "integer", "description": "Rows for audit/events (default 50)."},
+ "cursor": {"type": "integer", "description": "Event cursor for incremental reads."},
+ "format": {"type": "string", "enum": ["json", "prometheus"], "default": "json"},
+ },
+ },
+ "annotations": _ann("Bridge Ops, Metrics & Tunnel Control", read_only=False, destructive=False, idempotent=False),
```

```
+ },
]

TOOL_MAP = {t["name"]: t for t in TOOL_SPECS}
@@ -1717,6 +1865,14 @@ def is_tool_call_read_only(name: str, args: dict) -> bool:
return True
if name == "open_cloud_action" and args.get("action") in ("datastore_list", "datastore_get"):
return True
+ if name == "agent_bus" and args.get("action", "stats") in (
+ "stats", "agents", "inbox", "tasks", "thread", "feed", "status",
+ ):
+ return True
+ if name == "bridge_ops" and args.get("action", "status") in (
+ "status", "metrics", "audit", "audit_verify", "events", "tunnel",
+ ):
+ return True
return False


@@ -1894,6 +2050,12 @@ def execute_tool(client: BridgeClient, name: str, args: dict) -> dict:
dm = args.get("dm", "auto")
peer_id = args.get("peerId")

+ # Bridge-level tools never depend on a Studio peer.
+ if name == "agent_bus":
+ return client.bus_action(args)
+ if name == "bridge_ops":
+ return client.ops_action(args)
+
if name == "self_test":
return run_bridge_self_test(client, include_studio_smoke=args.get("includeStudioSmoke", True), timeout=args.get("timeout", 180))

@@ -2723,6 +2885,24 @@ RESOURCES = [
"description": "Service breakdown, instance census, script inventory + hashes, remotes, tags, and settings.",
"mimeType": "application/json",
},
+ {
+ "uri": "studio://bus",
+ "name": "Multi-Agent Bus (Presence, Tasks, Messages)",
+ "description": "Registered agents, open task board with leases, and the current activity cursor - shared by every agent on this bridge.",
+ "mimeType": "application/json",
+ },
+ {
+ "uri": "studio://ops",
+ "name": "Bridge Operations Report",
+ "description": "Queue depth, peer health, tunnel state, metrics, recent audit entries, and open tasks in one document.",
+ "mimeType": "application/json",
+ },
+ {
+ "uri": "studio://audit",
+ "name": "Tamper-Evident Audit Log (head)",
+ "description": "Latest hash-chained audit entries plus chain verification status.",
+ "mimeType": "application/json",
+ },
]

RESOURCE_TEMPLATES = [
@@ -2794,6 +2974,20 @@ def read_mcp_resource(client: BridgeClient, uri: str) -> dict:
if uri == "studio://survey":
data = execute_tool(client, "survey_place", {"includeSource": False})
```

```
return {"contents": [{"uri": uri, "mimeType": "application/json", "text": json.dumps(data, indent=2)}]}
+ if uri == "studio://bus":
+ data = client.bus_action({"action": "stats"})
+ board = client.bus_action({"action": "tasks", "includeDone": False})
+ roster = client.bus_action({"action": "agents"})
+ payload = {"stats": data, "agents": roster, "openTasks": board}
+ return {"contents": [{"uri": uri, "mimeType": "application/json", "text": json.dumps(payload, indent=2)}]}
+ if uri == "studio://ops":
+ data = client.ops_action({"action": "status"})
+ return {"contents": [{"uri": uri, "mimeType": "application/json", "text": json.dumps(data, indent=2)}]}
+ if uri == "studio://audit":
+ data = client.ops_action({"action": "audit", "limit": 25})
+ verify = client.ops_action({"action": "audit_verify"})
+ payload = {"entries": data, "verification": verify}
+ return {"contents": [{"uri": uri, "mimeType": "application/json", "text": json.dumps(payload, indent=2)}]}
if uri.startswith("studio://scripts/"):
path = uri[len("studio://scripts/") :]
data = execute_tool(client, "script_read", {"path": path})

```

roblox-bridge/arena_studio.py (diff)

203 lines

ops / metrics / audit / tunnel / tokens / bus CLI

```
diff --git a/roblox-bridge/arena_studio.py b/roblox-bridge/arena_studio.py
index 0d3e7d8..b156ce0 100644
--- a/roblox-bridge/arena_studio.py
+++ b/roblox-bridge/arena_studio.py
@@ -1,6 +1,6 @@
#!/usr/bin/env python3
"""
-arena_studio.py - Full-featured CLI for the Arena <-> Roblox Studio MCP Bridge (v4.0.0).
+arena_studio.py - Full-featured CLI for the Arena <-> Roblox Studio MCP Bridge (v5.0.0).

Supports every MCP tool and bridge operation from your terminal or HANDS MODE (`ab.ps1`),
with strict UTF-8 handling on Windows, macOS, and Linux.
@@ -20,6 +20,19 @@ Core commands:
python3 arena_studio.py sync ../nightloop/src ServerScriptService.NightLoop
python3 arena_studio.py run "return {place = game.Name}" [--dm edit|server|client]
python3 arena_studio.py runfile ../jobs/baseline_survey.lua
+
+v5 ops & multi-agent commands:
+ python3 arena_studio.py ops [--json]
+ python3 arena_studio.py metrics [--prometheus]
+ python3 arena_studio.py audit [--verify] [--limit 20]
+ python3 arena_studio.py tunnel status|start|stop|restart
+ python3 arena_studio.py tokens list|create --label ci --role write --ttl 86400|revoke --id tok-123
+ python3 arena_studio.py bus register --name "Gemini 4 Argon" --role partner
+ python3 arena_studio.py bus agents|tasks|feed|export
+ python3 arena_studio.py bus post --agent agent-xxxx --subject "handoff" --body "take the UI pass" --to agent-yyyy
+ python3 arena_studio.py bus task-create --title "Polish shop UI" --priority high --agent agent-xxxx
+ python3 arena_studio.py bus task-claim --task task-xxxx --agent agent-xxxx
+ python3 arena_studio.py bus task-complete --task task-xxxx --agent agent-xxxx --result "shipped"
"""

import argparse
@@ -78,6 +91,26 @@ def call(method: str, path: str, payload=None, timeout: float = 140.0):
return {"error": f"cannot reach bridge at {BASE} ({exc.reason}). Is server.py running?"}


+def call_text(method: str, path: str, payload=None, timeout: float = 60.0) -> str:
+ """Raw-text response helper (Prometheus exposition, SSE snapshots, markdown)."""
+ url = f"{BASE}{path}"
+ headers = {}
+ if TOKEN:
+ headers["X-Arena-Token"] = TOKEN
+ url += ("&" if "?" in url else "?") + f"token={TOKEN}"
+ data = json.dumps(payload).encode("utf-8") if payload is not None else None
+ if data is not None:
+ headers["Content-Type"] = "application/json"
+ req = urllib.request.Request(url, data=data, method=method, headers=headers)
+ try:
+ with urllib.request.urlopen(req, timeout=timeout) as resp:
+ return resp.read().decode("utf-8", "replace")
+ except urllib.error.HTTPError as exc:
+ return f"HTTP {exc.code}: " + exc.read().decode("utf-8", "replace")[:400]
+ except urllib.error.URLError as exc:
+ return f"cannot reach bridge at {BASE} ({exc.reason}). Is server.py running?"
+
+
def submit(kind: str, payload: dict, note: str = None, wait: float = 60.0, dm: str = "auto", peer_id: str = None):
body = {"type": kind, "payload": payload, "note": note, "wait": wait}
if dm and dm != "auto":
@@ -405,6 +438,48 @@ def main() -> int:
```

```
p = sub.add_parser("system", help="install a drop-in server+shared+client system from roblox-kit/systems")
p.add_argument("name", help="e.g. liveops/codes | liveops/playtime-gifts | monetization/offline-earnings | core/data | all")

+ p = sub.add_parser("ops", help="consolidated operations report (peers, queue, tunnel, bus, audit)")
+ p.add_argument("--json", action="store_true")
+
+ p = sub.add_parser("metrics", help="bridge counters, gauges and latency percentiles")
+ p.add_argument("--prometheus", action="store_true")
+
+ p = sub.add_parser("audit", help="read or verify the tamper-evident audit log")
+ p.add_argument("--verify", action="store_true")
+ p.add_argument("--limit", type=int, default=20)
+
+ p = sub.add_parser("tunnel", help="supervised public tunnel (cloudflared/ngrok)")
+ p.add_argument("action", nargs="?", default="status", choices=["status", "start", "stop", "restart"])
+
+ p = sub.add_parser("tokens", help="scoped access tokens (read/write/admin) & rate limits")
+ p.add_argument("action", nargs="?", default="list", choices=["list", "create", "revoke", "roles"])
+ p.add_argument("--label", default="cli-token")
+ p.add_argument("--role", default="write", choices=["read", "write", "admin"])
+ p.add_argument("--ttl", type=int, default=None, help="Token lifetime in seconds (default: never expires)")
+ p.add_argument("--rate", type=float, default=None, help="Requests per second (0 disables throttling)")
+ p.add_argument("--id", default=None, help="Token id to revoke")
+
+ p = sub.add_parser("bus", help="multi-agent coordination bus (presence, messages, tasks)")
+ p.add_argument("action", nargs="?", default="agents",
+ choices=["register", "agents", "inbox", "post", "tasks", "task-create", "task-claim",
+ "task-complete", "task-update", "feed", "export", "stats"])
+ p.add_argument("--agent", default=os.environ.get("ARENA_AGENT_ID", "cli-agent"))
+ p.add_argument("--name", default=None)
+ p.add_argument("--role", default="agent")
+ p.add_argument("--to", default=None)
+ p.add_argument("--subject", default="")
+ p.add_argument("--body", default="")
+ p.add_argument("--kind", default="note")
+ p.add_argument("--priority", default="normal")
+ p.add_argument("--task", default=None)
+ p.add_argument("--title", default="")
+ p.add_argument("--detail", default="")
+ p.add_argument("--result", default="")
+ p.add_argument("--status", default=None)
+ p.add_argument("--since", type=int, default=0)
+ p.add_argument("--limit", type=int, default=30)
+ p.add_argument("--out", default=None, help="Write the exported handoff markdown here")
+
p = sub.add_parser("import-audio", help="import a WAV/OGG/MP3 or built-in CC0 sound into Studio")
p.add_argument("source", help="local audio file path or repo audio filename (e.g. click_001.wav)")
p.add_argument("--target", default="SoundService")
@@ -844,6 +919,94 @@ def main() -> int:
"peerId": args.peer,
}, args.full, args.raw_json)

+ # ---------------------------- v5 ops & bus ----------------------------
+ if args.cmd == "ops":
+ data = call("GET", "/api/ops")
+ if args.json:
+ print(json.dumps(data, indent=2, ensure_ascii=False))
+ else:
+ h = data.get("health", {})
+ t = data.get("tunnel", {})
```

```
+ b = data.get("bus", {})
+ print(f"bridge v{data.get('version')} uptime {data.get('uptimeSeconds')}s")
+ print(f"studio : {'connected' if h.get('studio_connected') else 'waiting'} peers={len(h.get('peers', []))} queue={h.get('queued')}")
+ print(f"tunnel : {t.get('state')} {t.get('publicUrl') or '-'}")
+ print(f"agents : {b.get('agentsOnline', 0)} online / {b.get('agents', 0)} tasks open={sum(v for k, v in (b.get('taskCounts') or {}).items() if k in ('todo','in_progress','blocked')
)}")
+ rows = data.get("openTasks") or []
+ if rows:
+ print("\nopen tasks:")
+ for row in rows[:10]:
+ print(f" [{row['status']:11}] {row['priority']:8} {row['title']} ({row.get('assignee') or 'unclaimed'})")
+ audit = data.get("audit") or []
+ if audit:
+ print("\nrecent audit:")
+ for e in audit[:5]:
+ print(f" {e['seq']:>4} {e['action']:<22} {e['actor']:<20} {e['outcome']}")
+ return 0
+
+ if args.cmd == "metrics":
+ if args.prometheus:
+ print(call_text("GET", "/api/metrics?format=prometheus"))
+ return 0
+ data = call("GET", "/api/metrics")
+ print(json.dumps(data, indent=2, ensure_ascii=False))
+ return 0
+
+ if args.cmd == "audit":
+ path = "/api/audit?verify=1" if args.verify else f"/api/audit?limit={args.limit}"
+ print(json.dumps(call("GET", path), indent=2, ensure_ascii=False))
+ return 0
+
+ if args.cmd == "tunnel":
+ if args.action == "status":
+ print(json.dumps(call("GET", "/api/tunnel"), indent=2, ensure_ascii=False))
+ else:
+ print(json.dumps(call("POST", "/api/tunnel", {"action": args.action}, timeout=120), indent=2, ensure_ascii=False))
+ return 0
+
+ if args.cmd == "tokens":
+ payload = {"action": args.action}
+ if args.action == "create":
+ payload.update({"label": args.label, "role": args.role})
+ if args.ttl:
+ payload["ttlSeconds"] = args.ttl
+ if args.rate is not None:
+ payload["rateLimitPerSecond"] = args.rate
+ if args.action == "revoke":
+ if not args.id:
+ print("--id is required for revoke", file=sys.stderr)
+ return 2
+ payload["id"] = args.id
+ print(json.dumps(call("POST", "/api/tokens", payload), indent=2, ensure_ascii=False))
+ return 0
+
+ if args.cmd == "bus":
+ payload = {"action": args.action.replace("-", "_")}
+ payload["agentId"] = args.agent
+ mapping = {
+ "name": "name", "role": "role", "to": "to", "subject": "subject", "body": "body",
+ "kind": "kind", "priority": "priority", "title": "title", "detail": "detail",
```

```
+ "result": "result", "since": "since", "limit": "limit",
+ }
+ for attr, key in mapping.items():
+ value = getattr(args, attr, None)
+ if value not in (None, "", 0):
+ payload[key] = value
+ if args.task:
+ payload["taskId"] = args.task
+ if args.status:
+ payload["status"] = args.status
+ if args.out:
+ payload["path"] = args.out
+ result = call("POST", "/api/bus", payload)
+ if args.action == "export" and isinstance(result, dict) and result.get("markdown"):
+ print(result["markdown"])
+ if result.get("path"):
+ print(f"\n[written] {result['path']}", file=sys.stderr)
+ return 0
+ print(json.dumps(result, indent=2, ensure_ascii=False))
+ return 0
+
if args.cmd == "import-audio":
targs = {"targetPath": args.target, "peerId": args.peer}
if Path(args.source).is_file():

```

roblox-bridge/setup.py (diff)

136 lines

Supervised tunnel launch + URL watcher

```
diff --git a/roblox-bridge/setup.py b/roblox-bridge/setup.py
index 23c61af..77626b2 100644
--- a/roblox-bridge/setup.py
+++ b/roblox-bridge/setup.py
@@ -1,7 +1,7 @@
#!/usr/bin/env python3
"""
setup.py - One command to install, configure, and run the Arena <-> Roblox
-Studio Bridge & MCP Server (v4.0.0).
+Studio Bridge & MCP Server (v5.0.0).

What it does:
1. Creates or reuses a shared secret in bridge.token - and if bridge.token is
@@ -10,10 +10,10 @@ What it does:
Plugins directory so you NEVER hit HTTP 401 token mismatches!
2. Writes ArenaBridge.lua v4.0.0 into your OS Roblox Studio Plugins folder with
the port + token baked in (leaving the tracked repo source file clean).
- 3. Starts server.py v4.0.0 on :8077 (serving Studio polling, REST CLI, Web
+ 3. Starts server.py v5.0.0 on :8077 (serving Studio polling, REST CLI, Web
Dashboard, AND Streamable HTTP MCP at http://127.0.0.1:8077/mcp).
4. Optionally writes .mcp.json (`--write-mcp`) for Claude Code / Cursor / VS Code
- and/or opens a public cloudflared tunnel (`--tunnel`).
+ and/or opens a supervised cloudflared/ngrok tunnel (`--tunnel`).

Usage:
python3 setup.py # install plugin + run server & HTTP MCP
@@ -319,7 +319,13 @@ def shutdown(*_):
def main():
ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
ap.add_argument("--port", type=int, default=8077)
- ap.add_argument("--tunnel", action="store_true", help="Expose bridge + /mcp via cloudflared quick tunnel")
+ ap.add_argument("--tunnel", action="store_true", help="Expose bridge + /mcp via a supervised cloudflared/ngrok tunnel")
+ ap.add_argument("--tunnel-flavor", default="auto", choices=["auto", "cloudflared", "ngrok"],
+ help="Tunnel provider (default: auto-detect)")
+ ap.add_argument("--tunnel-token", default=None,
+ help="Cloudflare named-tunnel token (stable hostname instead of a random quick-tunnel URL)")
+ ap.add_argument("--public-url", default=None,
+ help="Report a tunnel URL you already run yourself (bridge will not manage it)")
ap.add_argument("--install-only", action="store_true", help="Install plugin (and optional --write-mcp) then exit")
ap.add_argument("--rotate", action="store_true", help="Mint a fresh shared secret token")
ap.add_argument("--read-only", action="store_true", help="Run server & MCP in Read-Only Inspector mode")
@@ -336,7 +342,7 @@ def main():

signal.signal(signal.SIGINT, shutdown)

- say(f"\n{BOLD}Arena <-> Roblox Studio MCP Bridge v4.0.0{OFF}")
+ say(f"\n{BOLD}Arena <-> Roblox Studio MCP Bridge v5.0.0{OFF}")
token = get_token(rotate=args.rotate, explicit_dir=args.plugins_dir, preferred_port=args.port)
say(f" token: {CYAN}{token}{OFF} {DIM}(stored in bridge.token){OFF}")

@@ -363,33 +369,61 @@ def main():
if args.place_id is not None:
server_cmd.extend(["--place-id", str(args.place_id)])

+ supervised_tunnel = bool(args.tunnel) and not args.public_url
+ if args.tunnel:
+ server_cmd.append("--tunnel")
+ server_cmd.extend(["--tunnel-flavor", args.tunnel_flavor])
+ if args.tunnel_token:
+ server_cmd.extend(["--tunnel-token", args.tunnel_token])
```

```
+ if args.public_url:
+ server_cmd.extend(["--public-url", args.public_url])
+
spawn(server_cmd, "bridge")
time.sleep(0.8)

- public_url = {"value": None}
-
- if args.tunnel:
- exe = find_binary("cloudflared")
- if exe:
- def got_url(url):
- if public_url["value"]:
- return
- public_url["value"] = url
- print()
- say("=" * 70, GREEN)
- say(" Paste these lines to Arena (or Streamable HTTP MCP client):", BOLD)
- say(f" URL: {url}", CYAN)
- say(f" MCP URL: {url}/mcp", CYAN)
- say(f" TOKEN: {token}", CYAN)
- say("=" * 70, GREEN)
- print()
-
- spawn([exe, "tunnel", "--url", f"http://localhost:{args.port}"], "tunnel", TUNNEL_RE, got_url)
- else:
- say("\n cloudflared not found. Install via:", YELLOW)
- say(" Windows: winget install --id Cloudflare.cloudflared", DIM)
- say(" macOS: brew install cloudflared", DIM)
- say(f" Or expose manually with: ngrok http {args.port}", DIM)
+ if args.public_url:
+ say("\n" + "=" * 70, GREEN)
+ say(" Public URL (externally managed tunnel):", BOLD)
+ say(f" URL: {args.public_url}", CYAN)
+ say(f" MCP URL: {args.public_url}/mcp", CYAN)
+ say(f" TOKEN: {token}", CYAN)
+ say("=" * 70, GREEN)
+ elif supervised_tunnel:
+ def watch_tunnel():
+ import urllib.request
+ announced = None
+ unavailable_reported = False
+ url = f"http://127.0.0.1:{args.port}/api/tunnel"
+ while True:
+ time.sleep(2.0)
+ try:
+ req = urllib.request.Request(url, headers={"X-Arena-Token": token})
+ with urllib.request.urlopen(req, timeout=5) as resp:
+ info = json.loads(resp.read().decode("utf-8"))
+ except Exception:
+ continue
+ state = info.get("state")
+ public = info.get("publicUrl")
+ if public and public != announced:
+ announced = public
+ print()
+ say("=" * 70, GREEN)
+ say(" Tunnel is live - paste these lines to Arena (or any MCP client):", BOLD)
+ say(f" URL: {public}", CYAN)
+ say(f" MCP URL: {public}/mcp", CYAN)
```

```
+ say(f" TOKEN: {token}", CYAN)
+ say(" The supervisor auto-restarts the tunnel and re-prints a new URL if it changes.", DIM)
+ say("=" * 70, GREEN)
+ print()
+ if state == "unavailable" and not unavailable_reported:
+ unavailable_reported = True
+ say("\n No tunnel binary found. Install one of:", YELLOW)
+ for hint in (info.get("installHints") or []):
+ say(f" {hint}", DIM)
+ say(f" Or expose manually with: ngrok http {args.port}", DIM)
+ say(" The bridge keeps running locally on 127.0.0.1.", DIM)
+
+ threading.Thread(target=watch_tunnel, daemon=True).start()

print()
say(f" Web Dashboard: http://127.0.0.1:{args.port}", GREEN)

```

roblox-bridge/test_bridge_mcp.py (diff)

385 lines

v5 test coverage (scopes, bus, reaping, audit, tunnel)

```
diff --git a/roblox-bridge/test_bridge_mcp.py b/roblox-bridge/test_bridge_mcp.py
index 7d1e453..67dc3e5 100644
--- a/roblox-bridge/test_bridge_mcp.py
+++ b/roblox-bridge/test_bridge_mcp.py
@@ -39,6 +39,7 @@ import unittest
import urllib.error
import urllib.request
from pathlib import Path
+from unittest import mock

HERE = Path(__file__).resolve().parent
TOOL_TEST_METRICS = {}
@@ -62,6 +63,298 @@ import server as bridge_server # noqa: E402
import setup as bridge_setup # noqa: E402


+class TestV5OpsPlatform(unittest.TestCase):
+ """v5: scoped tokens, rate limits, multi-agent bus, resilient jobs, audit chain, tunnel."""
+
+ def setUp(self):
+ self.tmp = tempfile.mkdtemp(prefix="arena-v5-")
+ self.port = _free_port()
+ self.base = f"http://127.0.0.1:{self.port}"
+ self.admin_token = "admin-secret-v5"
+ self.httpd, self.state = create_http_server(
+ host="127.0.0.1", port=self.port, token=self.admin_token, state_dir=self.tmp,
+ )
+ self.thread = threading.Thread(target=self.httpd.serve_forever, daemon=True)
+ self.thread.start()
+
+ def tearDown(self):
+ self.state._reaper_stop.set()
+ self.state.tunnel.stop()
+ self.httpd.shutdown()
+ self.httpd.server_close()
+
+ def _req(self, path, method="GET", body=None, token=None):
+ headers = {"Content-Type": "application/json"}
+ if token:
+ headers["Authorization"] = f"Bearer {token}"
+ data = json.dumps(body).encode("utf-8") if body is not None else None
+ req = urllib.request.Request(self.base + path, data=data, method=method, headers=headers)
+ try:
+ with urllib.request.urlopen(req, timeout=20) as resp:
+ raw = resp.read().decode("utf-8")
+ return resp.status, (json.loads(raw) if raw and raw.strip().startswith(("{", "[")) else raw)
+ except urllib.error.HTTPError as exc:
+ raw = exc.read().decode("utf-8")
+ return exc.code, (json.loads(raw) if raw.strip().startswith(("{", "[")) else raw)
+
+ # -- tokens / scopes / rate limiting -----------------------------------
+
+ def test_scoped_tokens_enforce_roles_and_rate_limits(self):
+ created = self.state.tokens.create("reader", role="read", rate_limit=100, burst=100)
+ reader = created["token"]
+ self.assertTrue(created["note"])
+ self.assertEqual(self._req("/api/state", token=reader)[0], 200)
+ self.assertEqual(self._req("/api/jobs", method="POST", body={"type": "ping"}, token=reader)[0], 403)
+ self.assertEqual(self._req("/api/tokens", token=reader)[0], 403)
+
```

```
+ # Read scope cannot invoke a mutating MCP tool, even over /mcp.
+ rpc = {"jsonrpc": "2.0", "id": 1, "method": "tools/call",
+ "params": {"name": "write_script", "arguments": {"path": "ServerScriptService.X", "source": "print(1)"}}}
+ status, body = self._req("/mcp", method="POST", body=rpc, token=reader)
+ self.assertEqual(status, 200)
+ self.assertTrue(body["result"]["isError"])
+ self.assertIn("insufficient_scope", json.dumps(body))
+
+ # Admin can list, mint a write token, and revoke it.
+ status, listing = self._req("/api/tokens", token=self.admin_token)
+ self.assertEqual(status, 200)
+ self.assertGreaterEqual(listing["count"], 2)
+ writer = self.state.tokens.create("writer", role="write", rate_limit=50)
+ status, jobs = self._req("/api/jobs", method="POST",
+ body={"type": "ping", "wait": 0}, token=writer["token"])
+ self.assertEqual(status, 201)
+ status, revoked = self._req("/api/tokens", method="POST",
+ body={"action": "revoke", "id": writer["id"]}, token=self.admin_token)
+ self.assertEqual(status, 200)
+ self.assertTrue(revoked["revoked"])
+ self.assertEqual(self._req("/api/jobs", method="POST", body={"type": "ping"}, token=writer["token"])[0], 401)
+
+ # Unknown tokens are rejected everywhere except the public health probe.
+ self.assertEqual(self._req("/api/state", token="nope")[0], 401)
+ self.assertEqual(self._req("/api/health")[0], 200)
+
+ # Rate limiting: one-request burst then 429 with Retry-After.
+ slow = self.state.tokens.create("firehose", role="read", rate_limit=0.01, burst=1)
+ self.assertEqual(self._req("/api/state", token=slow["token"])[0], 200)
+ status, body = self._req("/api/state", token=slow["token"])
+ self.assertEqual(status, 429)
+ self.assertIn("rate limit", body["error"])
+ self.assertGreaterEqual(body["retryAfter"], 1)
+
+ # -- multi-agent bus ------------------------------------------------
+
+ def test_agent_bus_presence_mailbox_leases_and_handoff(self):
+ res = self._req("/api/bus", method="POST", token=self.admin_token, body={
+ "action": "register", "name": "Gemini 4 Argon", "role": "partner",
+ "capabilities": ["roblox", "qa"],
+ })
+ self.assertEqual(res[0], 200)
+ gem = res[1]
+ claude = self.state.bus.handle({"action": "register", "name": "Claude", "role": "builder"})
+ self.assertEqual(self.state.bus.stats()["agents"], 2)
+
+ self.state.bus.handle({"action": "post", "agentId": gem["id"], "to": claude["id"],
+ "subject": "handoff", "body": "take the UI pass", "kind": "handoff"})
+ self.state.bus.handle({"action": "post", "agentId": gem["id"], "subject": "broadcast",
+ "body": "standup in 5"})
+ inbox = self.state.bus.handle({"action": "inbox", "agentId": claude["id"]})
+ self.assertEqual(inbox["count"], 2)
+ self.assertEqual(inbox["unread"], 2)
+ self.state.bus.handle({"action": "mark_read", "agentId": claude["id"], "all": True})
+ self.assertEqual(self.state.bus.handle({"action": "inbox", "agentId": claude["id"]})["unread"], 0)
+
+ task = self.state.bus.handle({"action": "task_create", "title": "Polish shop UI",
+ "createdBy": gem["id"], "priority": "critical", "tags": ["ui"]})
+ # Claim with a one-second lease; a second agent must be refused while it holds.
+ self.state.bus.handle({"action": "task_claim", "taskId": task["id"], "agentId": gem["id"], "leaseSeconds": 1})
```

```
+ with self.assertRaises(ValueError):
+ self.state.bus.handle({"action": "task_claim", "taskId": task["id"], "agentId": claude["id"]})
+ # ...and allowed once the lease expires, without any manual cleanup.
+ time.sleep(1.2)
+ taken = self.state.bus.handle({"action": "task_claim", "taskId": task["id"], "agentId": claude["id"]})
+ self.assertEqual(taken["assignee"], claude["id"])
+ self.assertEqual(taken["leaseExpiries"], 1)
+
+ done = self.state.bus.handle({"action": "task_complete", "taskId": task["id"], "agentId": claude["id"],
+ "result": "UI shipped", "artifacts": ["StarterGui/Shop"]})
+ self.assertEqual(done["status"], "done")
+ handoff = self.state.bus.handle({"action": "export", "path": os.path.join(self.tmp, "HANDOFF.md")})
+ self.assertIn("# Agent handoff brief", handoff["markdown"])
+ self.assertIn("Polish shop UI", handoff["markdown"])
+ self.assertTrue(os.path.isfile(handoff["path"]))
+
+ # -- resilient job queue ------------------------------------------------
+
+ def test_job_idempotency_priority_retry_and_reaping(self):
+ state = self.state
+ job1, _ = state.enqueue_job("ping", note="first", idempotency_key="abc123")
+ job2, _ = state.enqueue_job("ping", note="dedupe", idempotency_key="abc123")
+ self.assertEqual(job1["id"], job2["id"])
+ self.assertEqual(state.jobs[job1["id"]]["deduplicated"], 1)
+ self.assertEqual(len([j for j in state.jobs.values() if j["status"] == "queued"]), 1)
+
+ low, _ = state.enqueue_job("ping", note="low", priority=7)
+ high, _ = state.enqueue_job("ping", note="high", priority=0)
+ peer = state.record_poll({"peer": ["peer-priority"], "dm": ["edit"], "place": ["p"], "placeId": ["1"]})
+ first = state.poll_for_job(peer, wait_seconds=0.1)
+ self.assertEqual(first["id"], high["id"])
+ second = state.poll_for_job(peer, wait_seconds=0.1)
+ self.assertEqual(second["id"], job1["id"]) # priority 5 beats priority 7
+ third = state.poll_for_job(peer, wait_seconds=0.1)
+ self.assertEqual(third["id"], low["id"])
+
+ # Transient failure is retried once, then fails permanently.
+ retry_job, evt = state.enqueue_job("ping", note="retry", max_retries=1)
+ state.poll_for_job(peer, wait_seconds=0.1)
+ state.complete_job({"id": retry_job["id"], "ok": False, "error": "peer disconnected mid-job",
+ "peerId": "peer-priority"})
+ self.assertEqual(state.jobs[retry_job["id"]]["status"], "queued")
+ self.assertFalse(evt.is_set())
+ state.poll_for_job(peer, wait_seconds=0.1)
+ state.complete_job({"id": retry_job["id"], "ok": False, "error": "peer disconnected again",
+ "peerId": "peer-priority"})
+ self.assertEqual(state.jobs[retry_job["id"]]["status"], "error")
+ self.assertTrue(evt.is_set())
+ self.assertEqual(state.jobs[retry_job["id"]]["retries"], 1)
+
+ # A job claimed by a peer that vanishes is requeued by the reaper.
+ orphan, _ = state.enqueue_job("ping", note="orphan")
+ state.poll_for_job(peer, wait_seconds=0.1)
+ with state.lock:
+ state.jobs[orphan["id"]]["claimed"] = time.time() - 60
+ for pid in list(state.peers):
+ state.peers[pid]["lastPoll"] = 0
+ result = state.reap_jobs()
+ self.assertGreaterEqual(result["recovered"], 1)
+ self.assertEqual(state.jobs[orphan["id"]]["status"], "queued")
```

```
+
+ # Timeout reaping finalises a stuck running job.
+ state.clear_queue()
+ stuck, stuck_evt = state.enqueue_job("ping", note="stuck", timeout=1)
+ peer2 = state.record_poll({"peer": ["peer-stuck"], "dm": ["edit"], "place": ["p"], "placeId": ["1"]})
+ state.poll_for_job(peer2, wait_seconds=0.1)
+ with state.lock:
+ state.jobs[stuck["id"]]["claimed"] = time.time() - 30
+ state.reap_jobs()
+ self.assertEqual(state.jobs[stuck["id"]]["status"], "error")
+ self.assertIn("timeout", state.jobs[stuck["id"]]["error"])
+ self.assertTrue(stuck_evt.is_set())
+
+ # -- observability ------------------------------------------------------
+
+ def test_metrics_events_and_tamper_evident_audit(self):
+ self.state.enqueue_job("ping", note="metrics")
+ self.state.telemetry.metrics.observe("unit_latency_ms", 12.5, {"route": "/api/jobs"})
+ self.state.telemetry.metrics.observe("unit_latency_ms", 30.0, {"route": "/api/jobs"})
+ self.state.telemetry.events.emit("unit.event", {"hello": "world"})
+
+ self.state.tokens.create("audited", role="read")
+ status, metrics = self._req("/api/metrics", token=self.admin_token)
+ self.assertEqual(status, 200)
+ names = {c["name"] for c in metrics["counters"]}
+ self.assertIn("bridge_jobs_enqueued_total", names)
+ hist = next(h for h in metrics["histograms"] if h["name"] == "unit_latency_ms")
+ self.assertEqual(hist["count"], 2)
+ self.assertGreaterEqual(hist["p95"], hist["p50"])
+
+ status, text = self._req("/api/metrics?format=prometheus", token=self.admin_token)
+ self.assertEqual(status, 200)
+ self.assertIn("# TYPE bridge_jobs_enqueued_total counter", text)
+ self.assertIn('unit_latency_ms_count{route="/api/jobs"} 2', text)
+
+ status, events = self._req("/api/events?once=1&since=0&limit=20", token=self.admin_token)
+ self.assertEqual(status, 200)
+ self.assertTrue(any(e["kind"] == "unit.event" for e in events["events"]))
+
+ status, verify = self._req("/api/audit?verify=1", token=self.admin_token)
+ self.assertEqual(status, 200)
+ self.assertTrue(verify["ok"])
+ self.assertGreater(verify["entries"], 0)
+
+ # Tamper with an entry in memory: the chain verification must notice.
+ with self.state.telemetry.audit.lock:
+ victim = self.state.telemetry.audit.entries[-1]
+ victim["actor"] = "someone-else"
+ broken = self.state.telemetry.audit.verify()
+ self.assertFalse(broken["ok"])
+ self.assertIn("hash", broken["reason"])
+
+ def test_tunnel_refuses_to_publish_an_unauthenticated_bridge(self):
+ open_state = BridgeState(token="", read_only=False, state_dir=tempfile.mkdtemp(prefix="arena-open-"))
+ blocked = open_state.tunnel_action({"action": "start"})
+ self.assertEqual(blocked["state"], "blocked")
+ self.assertIn("without a token", blocked["lastError"])
+ # Status checks are always allowed.
+ self.assertIn(open_state.tunnel_action({"action": "status"})["state"], ("stopped", "unavailable"))
+ # With a token configured the guard steps aside (the sandbox has no binary,
```

```
+ # so the supervisor reports unavailable rather than blocked).
+ allowed = self.state.tunnel_action({"action": "start"})
+ self.assertNotEqual(allowed["state"], "blocked")
+
+ def test_ops_report_and_public_health_shape(self):
+ status, health = self._req("/api/health")
+ self.assertEqual(status, 200)
+ self.assertEqual(health["version"], "5.0.0")
+ self.assertIn("tunnel", health)
+ self.assertIn("agents", health)
+ status, ops = self._req("/api/ops", token=self.admin_token)
+ self.assertEqual(status, 200)
+ for key in ("health", "tunnel", "bus", "audit", "recentJobs", "openTasks"):
+ self.assertIn(key, ops)
+
+ # -- tunnel supervisor ------------------------------------------------
+
+ def test_tunnel_supervisor_scrapes_url_and_restarts(self):
+ from tunnel_manager import TunnelManager
+ fake = os.path.join(self.tmp, "fake-cloudflared")
+ with open(fake, "w", encoding="utf-8") as fh:
+ fh.write(
+ "#!/usr/bin/env python3\n"
+ "import sys, time\n"
+ "print('INF Requesting new quick Tunnel...', flush=True)\n"
+ "print('INF | https://unit-test-tunnel.trycloudflare.com |', flush=True)\n"
+ "time.sleep(30)\n"
+ )
+ os.chmod(fake, 0o755)
+ tunnel = TunnelManager(port=self.port, binary=fake, flavor="cloudflared",
+ state_dir=self.tmp, health_interval=3600)
+ try:
+ started = tunnel.start()
+ self.assertIn(started["state"], ("starting", "online"))
+ for _ in range(60):
+ if tunnel.status()["state"] == "online":
+ break
+ time.sleep(0.1)
+ status = tunnel.status()
+ self.assertEqual(status["state"], "online")
+ self.assertEqual(status["publicUrl"], "https://unit-test-tunnel.trycloudflare.com")
+ self.assertEqual(status["mcpEndpoint"], "https://unit-test-tunnel.trycloudflare.com/mcp")
+ self.assertTrue(status["running"])
+ stopped = tunnel.stop()
+ self.assertEqual(stopped["state"], "stopped")
+ finally:
+ tunnel.stop()
+
+ # Named Cloudflare tunnels use `run --token`, never a quick-tunnel `--url`.
+ named = TunnelManager(port=self.port, binary=fake, flavor="cloudflared",
+ named_token="cf-named-token", state_dir=self.tmp)
+ named._detect()
+ cmd = named._command()
+ self.assertIn("run", cmd)
+ self.assertIn("--token", cmd)
+ self.assertNotIn("--url", cmd)
+ self.assertNotIn("http://127.0.0.1", " ".join(cmd))
+ self.assertTrue(named.status()["named"])
+
+ # No binary -> explicit unavailable state with install hints, never a crash.
```

```
+ with mock.patch("tunnel_manager.find_tunnel_binary", return_value=(None, None)):
+ missing = TunnelManager(port=self.port, state_dir=self.tmp)
+ result = missing.start()
+ self.assertEqual(result["state"], "unavailable")
+ self.assertIn("cloudflared", result["lastError"])
+ self.assertTrue(result["installHints"])
+
+
def _free_port() -> int:
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
s.bind(("127.0.0.1", 0))
@@ -141,14 +434,14 @@ class TestV4RegressionFixtures(unittest.TestCase):
self.assertFalse(any(x["code"] == "reset_on_spawn_enabled" for x in report["issues"]))

def test_missing_live_peer_fast_fails(self):
- state = BridgeState()
+ state = BridgeState(state_dir=tempfile.mkdtemp(prefix="arena-state-"))
client = BridgeClient(direct_dispatch=state.direct_dispatch)
job = client.submit_job("inspect", {"path": "Workspace"}, wait=60, dm="edit")
self.assertEqual(job["status"], "error")
self.assertEqual(job["reason"], "no_peer")

def test_offline_renderer_accepts_exported_tree_without_peer(self):
- state = BridgeState()
+ state = BridgeState(state_dir=tempfile.mkdtemp(prefix="arena-state-"))
client = BridgeClient(direct_dispatch=state.direct_dispatch)
tree = {"name": "Workspace", "className": "Workspace", "children": [{
"name": "OfflinePart", "className": "Part", "structuredProperties": {
@@ -189,9 +482,9 @@ class TestV4RegressionFixtures(unittest.TestCase):
from unittest.mock import patch
with tempfile.TemporaryDirectory() as folder:
with patch.object(bridge_server, "SNAPSHOT_DIR", Path(folder)):
- first = BridgeState()
+ first = BridgeState(state_dir=folder)
saved = first.handle_snapshots({"action": "save", "label": "named-baseline", "root": "ServerStorage", "scripts": [{"path": "ServerStorage.A", "source": "return 1"}]})
- second = BridgeState()
+ second = BridgeState(state_dir=folder)
got = second.handle_snapshots({"action": "get", "id": saved["id"]})
self.assertEqual(got["label"], "named-baseline")
self.assertEqual(got["scripts"][0]["path"], "ServerStorage.A")
@@ -238,11 +531,13 @@ class TestEndToEndBridgeAndMCP(unittest.TestCase):
cls.port = _free_port()
cls.token = "test-secret-token-v3"
cls.base_url = f"http://127.0.0.1:{cls.port}"
+ cls.state_dir = tempfile.mkdtemp(prefix="arena-e2e-state-")
cls.httpd, cls.state = create_http_server(
host="127.0.0.1",
port=cls.port,
token=cls.token,
read_only=False,
+ state_dir=cls.state_dir,
)
cls.server_thread = threading.Thread(target=cls.httpd.serve_forever, daemon=True)
cls.server_thread.start()
@@ -356,10 +651,29 @@ class TestEndToEndBridgeAndMCP(unittest.TestCase):
], auth_mode="query")
self.assertEqual(st_b, 200)
self.assertEqual(len(batch_resp), 3)
- self.assertEqual(len(batch_resp[1]["result"]["resources"]), 5)
+ self.assertEqual(len(batch_resp[1]["result"]["resources"]), 8)
```

```
self.assertEqual(len(batch_resp[2]["result"]["prompts"]), 4)

def test_02_all_core_studio_tools_end_to_end(self):
+ # 0. v5 bridge-level tools (no Studio peer required)
+ reg = self.call_tool("agent_bus", {"action": "register", "name": "pytest-agent", "role": "qa"})
+ self.assertTrue(reg["online"])
+ posted = self.call_tool("agent_bus", {"action": "post", "agentId": reg["id"], "subject": "test", "body": "hello bus"})
+ self.assertTrue(posted["id"].startswith("msg-"))
+ task = self.call_tool("agent_bus", {"action": "task_create", "title": "v5 smoke", "agentId": reg["id"], "priority": "high"})
+ self.assertEqual(task["status"], "todo")
+ claimed = self.call_tool("agent_bus", {"action": "task_claim", "taskId": task["id"], "agentId": reg["id"]})
+ self.assertEqual(claimed["status"], "in_progress")
+ done = self.call_tool("agent_bus", {"action": "task_complete", "taskId": task["id"], "agentId": reg["id"], "result": "ok"})
+ self.assertEqual(done["status"], "done")
+ ops = self.call_tool("bridge_ops", {"action": "status"})
+ self.assertEqual(ops["version"], "5.0.0")
+ self.assertIn("tunnel", ops)
+ metrics = self.call_tool("bridge_ops", {"action": "metrics"})
+ self.assertIn("counters", metrics)
+ verify = self.call_tool("bridge_ops", {"action": "audit_verify"})
+ self.assertTrue(verify["ok"])
+
# 1. Session & Peers
st = self.call_tool("get_studio_state")
self.assertTrue(st["studioConnected"])

```

roblox-bridge/mcp_server.py (diff)

13 lines

Version banner

```
diff --git a/roblox-bridge/mcp_server.py b/roblox-bridge/mcp_server.py
index ddbd02b..47b8b73 100644
--- a/roblox-bridge/mcp_server.py
+++ b/roblox-bridge/mcp_server.py
@@ -1,7 +1,7 @@
#!/usr/bin/env python3
"""
mcp_server.py - Standalone Model Context Protocol (MCP) stdio server for
-Roblox Studio (v4.0.0).
+Roblox Studio (v5.0.0).

Works out-of-the-box with Claude Code, Claude Desktop, Cursor, Codex CLI,
VS Code Copilot, Windsurf, Zed, Cline, and any standard MCP host.

```

roblox-bridge/ab.ps1 (diff)

22 lines

Version banner + ops examples

```
diff --git a/roblox-bridge/ab.ps1 b/roblox-bridge/ab.ps1
index 0391b83..4cf9341 100644
--- a/roblox-bridge/ab.ps1
+++ b/roblox-bridge/ab.ps1
@@ -1,4 +1,4 @@
-# HANDS MODE & CLI helper for Windows PowerShell (v4.0.0).
+# HANDS MODE & CLI helper for Windows PowerShell (v5.0.0).
# Reads BRIDGE_TOKEN from bridge.token next to this script (or auto-recovers
# from installed ArenaBridge.lua in %LOCALAPPDATA%\Roblox\Plugins!) so tokens
# never appear in shell history. Enforces UTF-8 so em-dashes & symbols are never
@@ -12,6 +12,11 @@
# .\ab.ps1 drift ..\nightloop\src ServerScriptService.NightLoop
# .\ab.ps1 sync ..\nightloop\src ServerScriptService.NightLoop
# .\ab.ps1 runfile ..\jobs\baseline_survey.lua
+# .\ab.ps1 ops # peers + queue + tunnel + agents + tasks
+# .\ab.ps1 tunnel status
+# .\ab.ps1 bus tasks
+# .\ab.ps1 metrics --prometheus
+# .\ab.ps1 audit --verify

param([Parameter(ValueFromRemainingArguments = $true)][string[]]$Cmd)


```

6. Verification snapshot

The v5 suite exercises the new surface end-to-end against the stateful mock Studio and a fake tunnel binary. Live Studio acceptance (real peer + real public tunnel) is still open and is tracked as G7/G8 in the task PDF.

```
36b76d5 handoff: add per-task scope lock row to the task pack
 handoff/Gemini-4-Argon-Tasks.pdf          |   54 +-
 handoff/Tunneling-v5-Code.pdf             | 1197 ++++++----------------------
 roblox-bridge/tools/build_handoff_pdfs.py |   14 +
 3 files changed, 267 insertions(+), 998 deletions(-)
```